

为什么需要RAG(检索增强生成)

1. 问答 基于上下文的信息理解与推理

要求模型：先理解问题，再去长文本里找到相关信息，最后提取或生成答案。

阅读理解：

The first recorded travels by Europeans to China and back date from this time. The most famous traveler of the period was the Venetian Marco Polo, whose account of his trip to "Cambaluc," the capital of the Great Khan, and of life there astounded the people of Europe. The account of his travels, Il milione (or, The Million, known in English as the Travels of Marco Polo), appeared about the year 1299. Some argue over the accuracy of Marco Polo's accounts due to the lack of mentioning the Great Wall of China, tea houses, which would have been a prominent sight since Europeans had yet to adopt a tea culture, as well the practice of foot binding by the women in capital of the Great Khan. Some suggest that Marco Polo acquired much of his knowledge through contact with Persian traders since many of the places he named were in Persian.

How did some suspect that Polo learned about China instead of by actually visiting it?

Answer: through contact with Persian traders

(Rajpurkar 等, 2016): SQuAD: 10万余道机器文本理解题

多模态问答问答



How many slices of pizza are there?
Is this a vegetarian pizza?

how many slices of pizza are there? is this a vegetarian pizza?

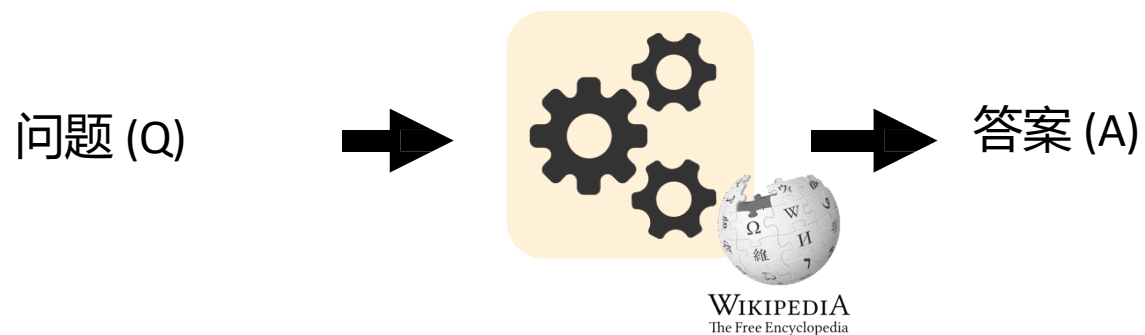


ChatGPT

There are 8 slices of pizza in the image. As for whether it's a vegetarian pizza, it appears to have toppings that could be vegetables, such as pieces that look like diced tomatoes and possibly green peppers. However, without a clearer image or a list of ingredients, it's impossible to be certain if there are no meat products on the pizza. Vegetarian pizzas typically do not include meat, but they can include a variety of vegetable toppings like onions, peppers, mushrooms, olives, and more. If there is any meat or meat-based toppings visible, then it would not be a vegetarian pizza.

开放式问答

现实世界里，我们根本不知道文章在哪。

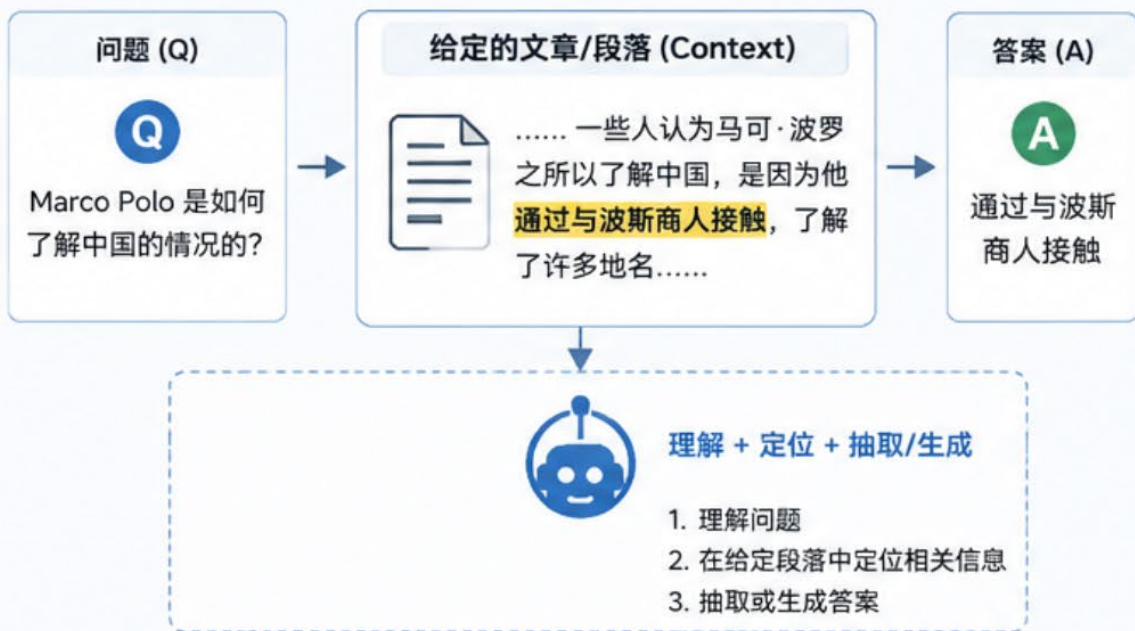


- 与阅读理解不同，我们并不假设存在给定的文章。
- 相反，我们只能访问大量文档（例如维基百科）。我们不知道答案位于何处，而目标是针对任何开放域问题返回答案。
- 这确实是一个更具挑战性且更贴近实际的问题！

这与处理特定领域（如医学、技术支持）问题的**寿采弄**系统形成鲜明对比。

阅读理解问答 (Reading Comprehension)

假设：答案已经存在于给定的文章/段落中



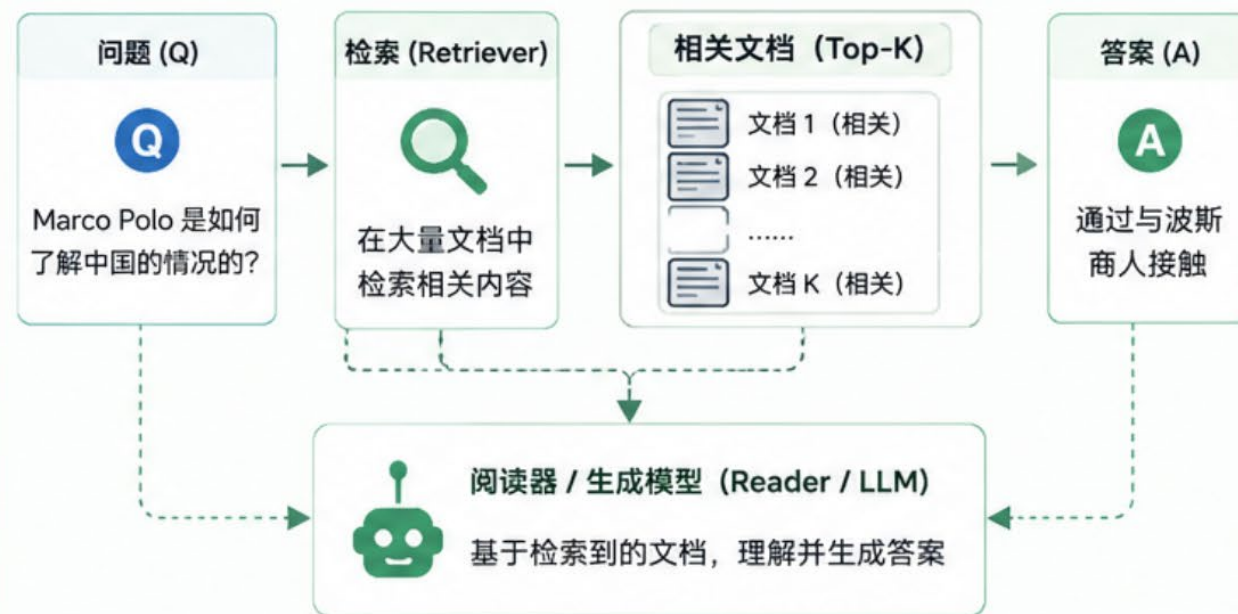
特点

- 只给定一段 (通常较短的) 文章/段落
- 答案一定在文章中
- 主要挑战: 理解文本并定位答案 (定位/推理)
- 典型数据集: SQuAD, Natural Questions (KILT) 等



开放式问答 (Open-domain QA)

假设：答案可能存在于海量文档中，需要先检索再回答



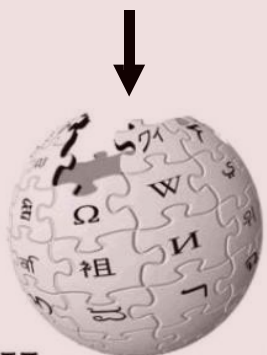
特点

- 不提供文章, 需要从海量文档 (如维基百科、网页等) 中检索
- 答案可能分散在多个文档中, 甚至需要综合信息
- 主要挑战: 检索的召回与排序质量 + 阅读理解 + 生成/推理
- 典型系统: Retriever-Reader、RAG、DrQA、DPR 等



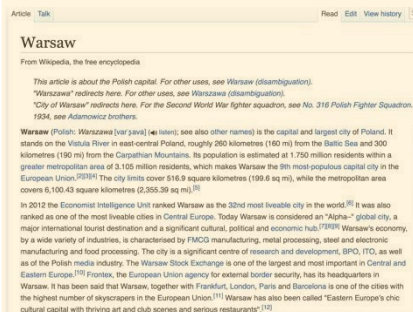
检索增强

1933年，华沙有多少居民说波兰语？



WIKIPEDIA

文档检索器



文档阅读器



833,500



<https://github.com/facebookresearch/DrQA>

Chen等人，2017年。通过阅读维基百科回答开放领域问题

语言模型的局限

- LLM 好像“知道很多东西”
- 但它真的记住了整个世界吗？

- 如果知识发生变化怎么办？

- 例如：

- 今天的新闻

- 最新论文

- 公司最新产品

- 当前股价

只靠参数记忆的问题

1. 知识会过时

- 新闻
- 法律
- 医疗知识
- 公司信息

2. 更新成本高

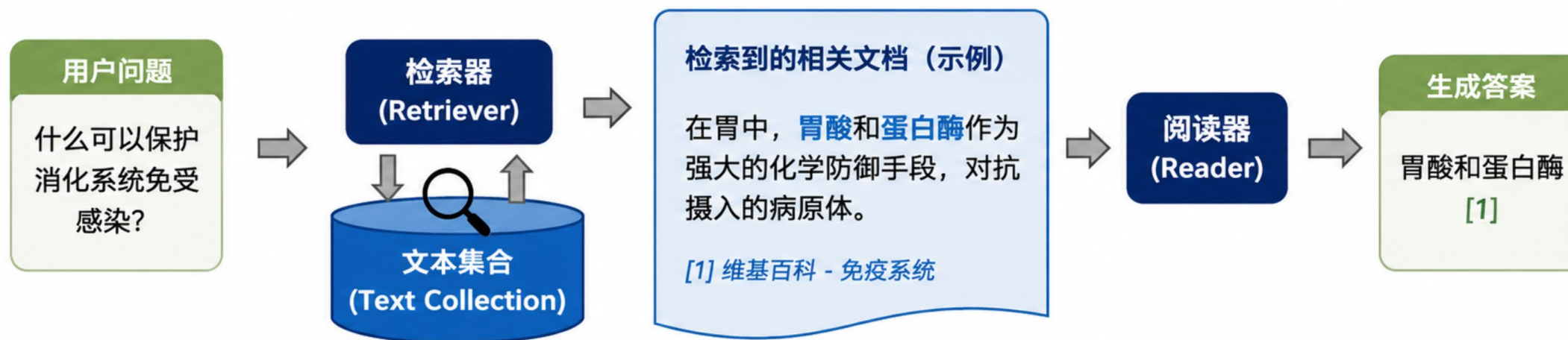
- 重新训练模型很昂贵
- 需要大量 **GPU** 和时间

3. 容易幻觉（Hallucination）

- 可能编造事实
- 无法保证来源正确

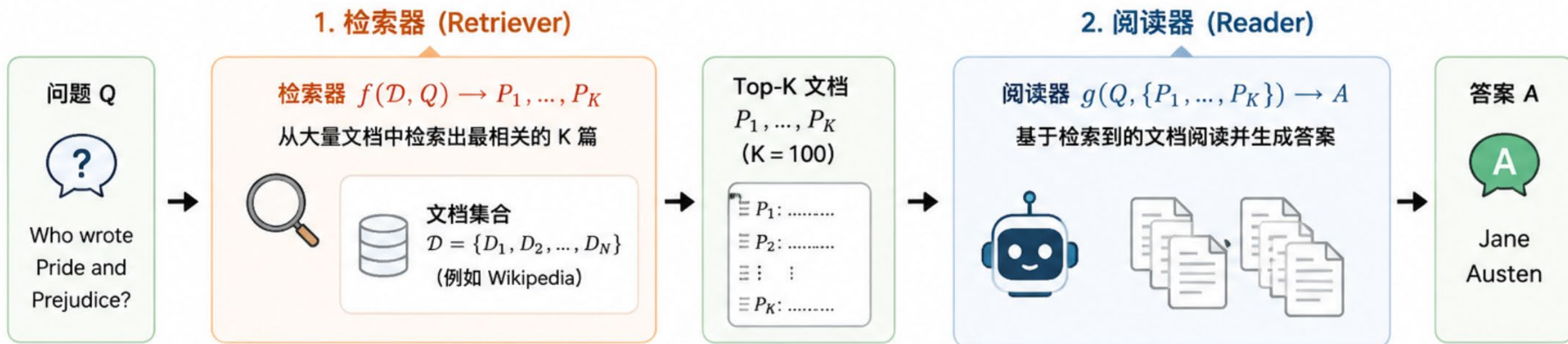
利用检索技术克服语言模型的不足

- 与其要求语言模型记住所有内容，我们能否在恰当的时机为其提供相关且有用的内容？
- 检索/搜索是识别此类相关信息的常用机制。检索系统的**两个核心优势**：
 - **动态性**：可轻松更新/向检索系统添加文档
 - **可解释性**：LM能够生成指向检索文档的指针，从而支持人工对其生成的内容（引文）进行验证



Retriever-Reader 框架

- 输入：一大批文档 $\mathcal{D} = D_1, D_2, \dots, D_N$ 以及问题 Q
- 输出：一个答案字符串 A



检索器 (Retriever)

一种标准的 TF-IDF 信息检索稀疏模型 (固定模块)

- 计算问题 Q 与每个文档的相关性分数 (如 TF-IDF)
- 返回最相关的 Top-K 文档 $P_1, \dots, P_K$

阅读器 (Reader)

一个神经阅读理解模型

- 可基于 SQuAD 及其他远端监督问答数据集进行训练
- 或直接使用零样本大型语言模型 (如 ChatGPT 等)

RAG= 检索 (Search) + 生成 (Generation)

让模型先从大量文档中检索相关信息，再基于这些信息生成答案。

传统 LLM vs RAG

传统 LLM



所有知识都存在
在模型参数里

RAG



通用能力在模型里，
事实知识在外部文档库

1. 输入问题

什么可以保护
消化系统避免
感染？



用户提问

2. 检索器 (Retriever) 从大量文档中找相关内容



文档库

(维基百科、书籍、网页等)

找到的 Top-K 文档

D1: 胃酸和蛋白酶可以杀死
摄入 的病原体，保护胃肠道。

D2: 免疫系统包含多种防御
机制，如白细胞、抗体等。

D3: 消化系统包括口腔、食道、
胃、 小肠、大肠等器官。

非参数化知识 (外部知识库)

可以随时更新，无需重新训练模型

3. 生成器 (Generator / LLM) 阅读检索结果并生成答案

将问题 + 检索到的文档
作为上下文输入大模型

问题

+

Top-K 文档

指令 (如: 请根据以上
信息回答问题)



大语言模型
(LLM)

参数化能力 (模型内部能力)

负责理解、推理和生成自然语言

4. 生成答案

胃酸和蛋白酶通过杀死
摄入的病原体来保护消
化系统避免感染。

(答案可包含引用来源)

[1] 维基百科 - 免疫系统

不同类型的检索器

我们应该如何为检索系统检索相关段落？

- 词重叠 (BM25)
- 向量检索 (DPR、句子向量)
- 其他，神经网络系统 (ColBERT)

快速检索器与慢速检索器: 不同的相关性计算方式

给定一个查询(Query)和大量文档(Document), 如何判断哪个文档最相关?

不同方法的本质区别: 计算“查询”和“文档”的相关性时, 考虑的细节越多, 效果越好, 但速度越慢。

1. 稀疏检索 (关键词匹配)

代表: TF-IDF / BM25



优点:

非常快
可扩展性好



缺点:

只看词面,
不理解语义

示例: car 与 automobile 无法匹配

2. 密集向量检索 (Embedding)

代表: DPR、Contriever 等



优点:

理解语义, 能匹配
不同表达的意思



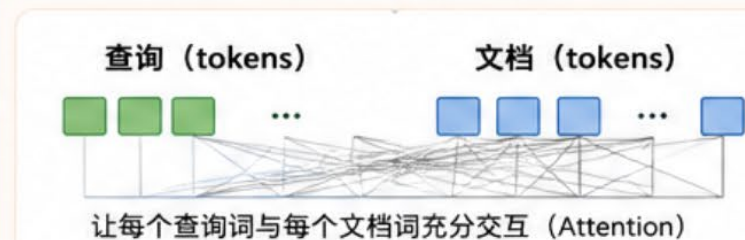
缺点:

比关键词匹配慢
需要预计算向量

示例: car 与 automobile 可以匹配

3. 交叉编码器 (Cross-Encoder)

代表: BERT Cross-Encoder 等



优点:

最准确, 能深度理解
复杂语义关系



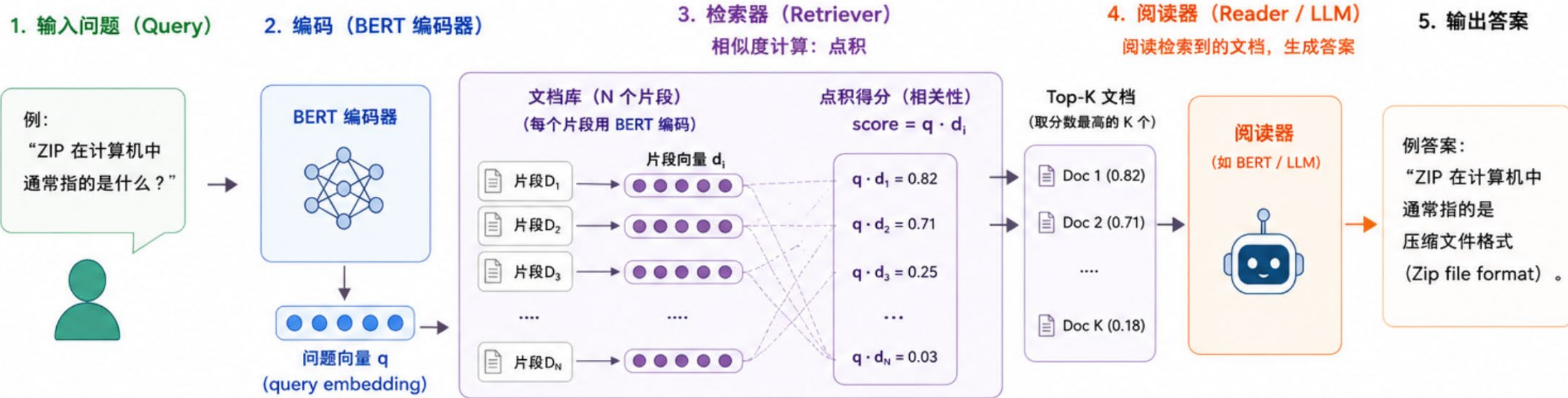
缺点:

非常慢, 计算量大
不易扩展

示例: 理解长距离依赖、否定关系等

我们可以训练检索器！ 阅读器与检索器的联合训练

让检索器学会找到“真正有助于回答问题”的文档，而不是仅仅看起来相关的文档。



关键点：所有文本片段都可以用 BERT 编码为向量，检索器的得分就是“问题向量 q ”与“片段向量 d_i ”的点积 ($q \cdot d_i$)。分数越高，越相关。

联合训练 (Joint Training)

根据阅读器的最终回答是否正确，反向优化检索器和阅读器，使其更容易找到真正有用的文档并正确回答问题。



问题：我们可以使用多少份文档？

检索器是关键——如果我们只能使用一份文档，那么就必须确保它准确无误。为什么不获取大量文档并将其传递给语言模型呢？

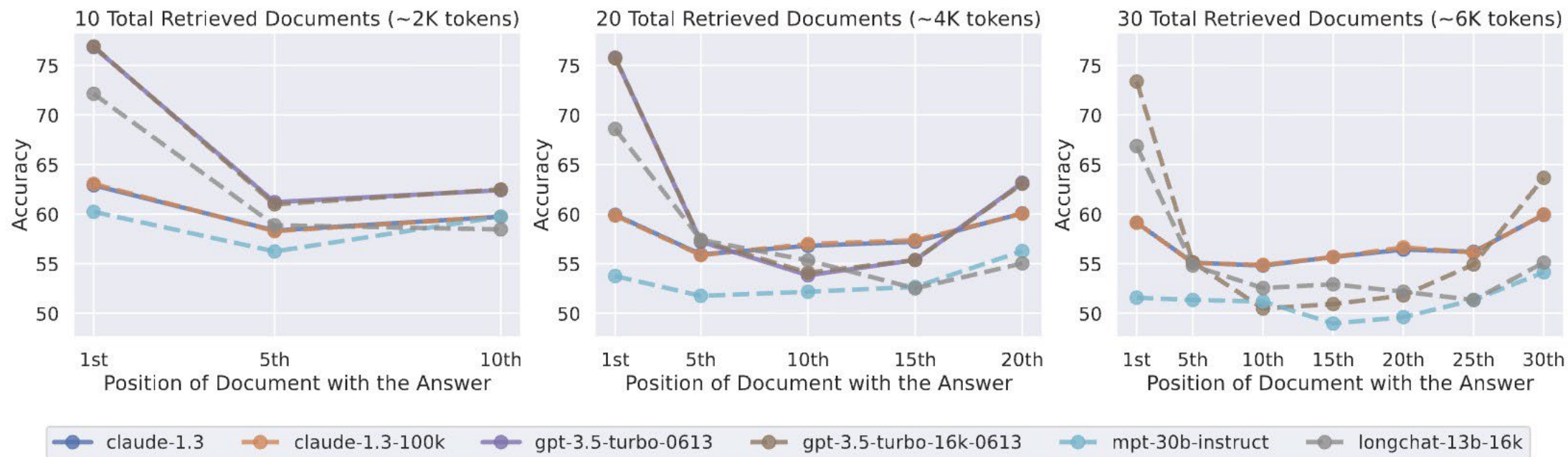
检索器只能找到相关文档，但如果传给模型的文档太少或太多，都会影响答案的准确性。



大型语言模型无法关注整个上下文 [Liu+ 2023]

LLM 能看到长上下文 $\neq$ LLM 能有效利用长上下文

长上下文问题困扰着你——大型语言模型无法很好地关注上下文！ 设置：1份
相关文档，其余均为无关文档



当相关文档位置变靠后模型性能明显下降。也就是说：模型并不会真正“阅读全部上下文”。

最佳闭卷表现：GPT-3.5-Turbo，约56%

最佳 Oracle（仅输入相关文档）性能：GPT-3.5-Turbo，约 88.5%

实际情况是：大型语言模型无法处理大量文档



直观结论：找到的文档越多（Recall 越高），不代表最终回答效果越好。 关键是：让 LLM **更容易聚焦**到真正相关的信息！

① 从检索器（Retriever）视角

返回的文档越多，Recall（召回率）越高

检索数量
增加



Recall
越来越高



5 篇文档



Recall
较低

可能没有找到答案



20 篇文档

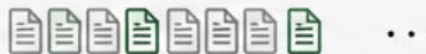


Recall
较高

大概率能找到答案



50 篇文档



Recall
非常高

几乎一定包含答案



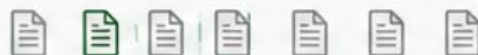
Retriever 会认为：返回越多越好！

② 从大语言模型（LLM）视角

文档太多会带来“信息过载”，反而更难回答

Case A: 适量文档（例如 10 篇）

相关文档更集中



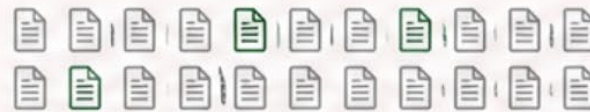
3 篇相关



LLM 容易聚焦在相关信息上
更容易给出正确答案

Case B: 大量文档（例如 50 篇）

相关文档被淹没在大量无关信息中



仍然只有 3 篇相关

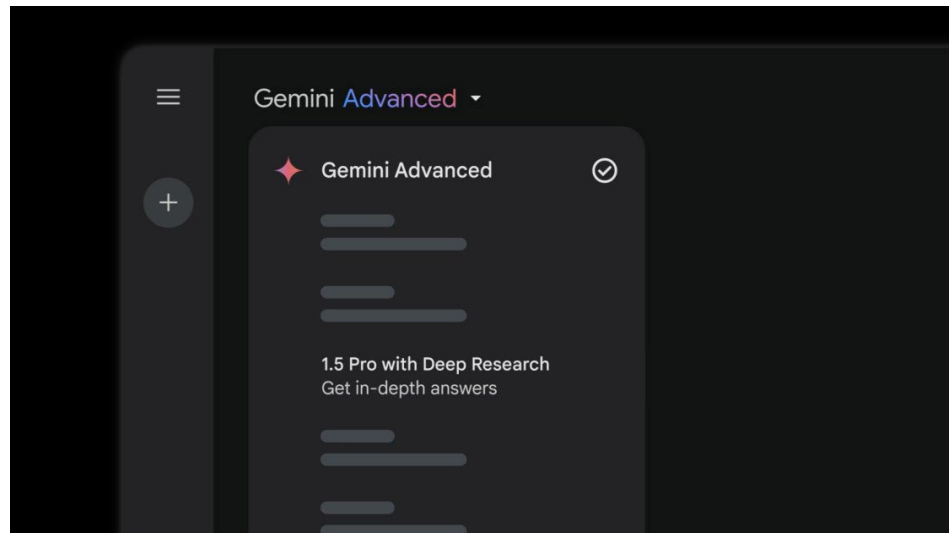
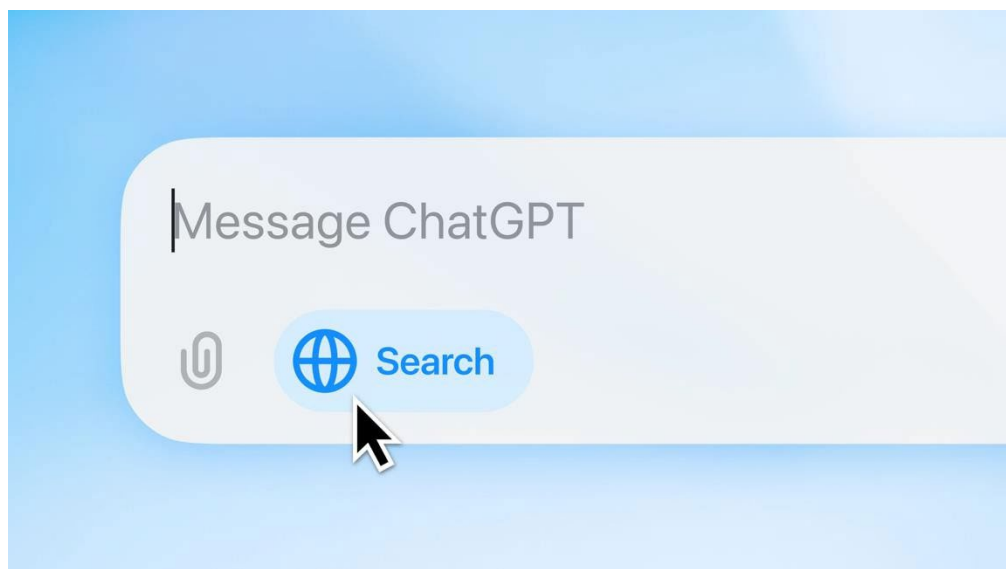
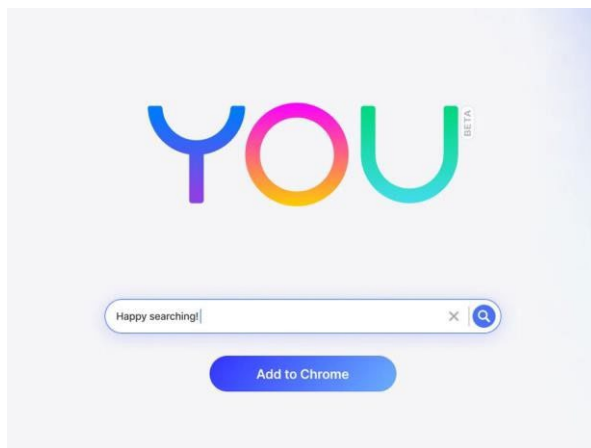
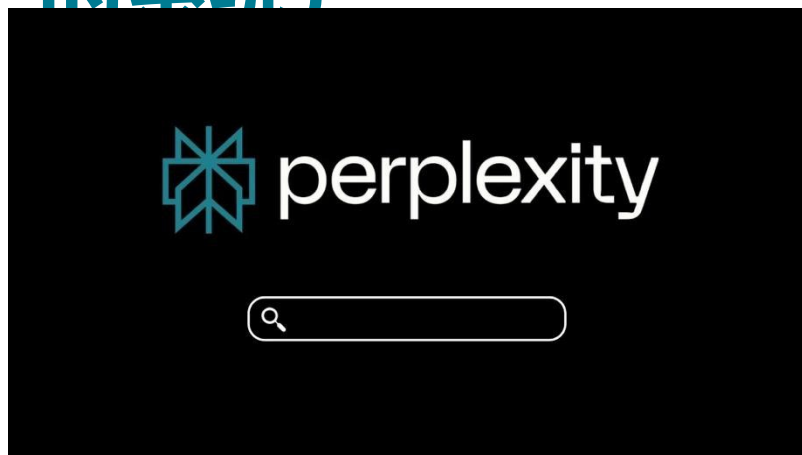


LLM 很难找到并聚焦相关信息
更容易回答错误或不确定



LLM 会认为：文档太多，反而更难！

具备网络搜索功能的大型语言模型（能够搜索并引用网络内容的系统）



大型语言模型的引用与幻觉

RAG的独特优势： 引用来源

这些引用本身是由LLM生成的

因此引用内容 *也可能出现幻觉*.....这种情况发生得有多频繁? 🙋

🔍 Create an itinerary for exploring Toronto over a span of two days. 🗣️ 📷

 Toronto is a vibrant and diverse city with many attractions and activities to suit different tastes and preferences. Here are some suggestions for creating an itinerary for exploring Toronto over two days:

- **Day 1:** Start your day with a visit to the **CN Tower**, the iconic landmark that offers panoramic views of the city and beyond ¹. You can also experience the thrill of walking on the glass floor or the EdgeWalk, the world's highest hands-free walk ¹. Next, head to the nearby **St. Lawrence Market**, one of the world's best food markets, where you can sample a variety of cuisines and local specialties ². After lunch, take a stroll along **Queen West**, a trendy neighborhood with eclectic shops, galleries, cafes and street art ³. In the evening, enjoy a show at one of the many theaters or comedy clubs in the **Entertainment District**, or catch a game at the **Scotiabank Arena** if you're a sports fan.

- **Day 2:** Explore the history and culture of Toronto at the **Royal Ontario Museum**, one of the largest and most comprehensive museums in North America, featuring exhibits on art, natural history, world cultures and more ⁴. Then, hop on a ferry to the **Toronto Islands**, a group of islands that offer a relaxing escape from the city, with beaches, parks, trails and amusement rides ³ ⁵. You can also rent a bike or kayak to explore the islands at your own pace. For dinner, head to **Chinatown**, one of the largest and most vibrant in North America, where you can find a variety of Asian cuisines and shops ³.

I hope this helps you plan your trip to Toronto. Have fun! 😊

Learn more:

- 1. [cntower.ca](https://www.cntower.ca)
- 2. [travel.usnews.com](https://www.travel.usnews.com)
- 3. [bing.com](https://www.bing.com)
- 4. [rom.on.ca](https://www.rom.on.ca)
- 5. [tripadvisor.com](https://www.tripadvisor.com)

大型语言模型的引用与幻觉

RAG的独特优势： 引用来源

这些引用本身是由LLM生成的

因此引用内容 *也可能出现幻觉* 这种情况发生频率如何?



Citation Precision: 模型给出的 citation 中，有多少是真的支持答案的？
Citation Recall: 所有“应该被引用”的证据里，模型到底引用到了多少。

输出内容易于阅读 / 在评分者看来具有实用性 (1-5分制)

<i>Perceived Utility</i> (↑)		<i>Fluency</i> (↑)	
Average Over All Queries		Average Over All Queries	
Bing Chat	4.34	Bing Chat	4.40
NeevaAI	4.48	NeevaAI	4.43
perplexity.ai	4.56	perplexity.ai	4.51
YouChat	4.62	YouChat	4.59
Average	4.50	Average	4.48

但准确率和召回率都很低..... 引用存在 ≠ 引用正确

<i>Citation Precision</i> (%; ↑)		<i>Citation Recall</i> (%; ↑)	
Average Over All Queries		Average Over All Queries	
Bing Chat	89.5	Bing Chat	58.7
NeevaAI	72.0	NeevaAI	67.6
perplexity.ai	72.7	perplexity.ai	68.7
YouChat	63.6	YouChat	11.1
Average	74.5	Average	51.5

引用幻觉的缓解方法

方法	基本思路
限制引用范围	只能引用检索到的资料，减少“乱引用”
引用检查	自动检查 citation 是否真的支持答案
更细粒度引用	让每句话甚至每个短语都对应证据
Self-RAG	模型生成时自己判断“有没有依据”
多轮检索	多查几次、交叉验证不同来源

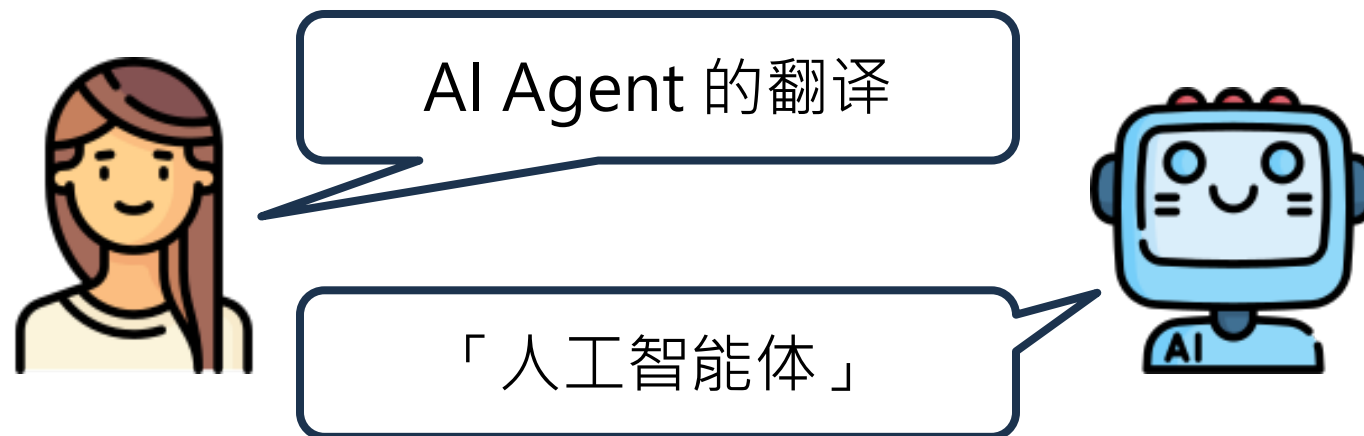
AI Agent

AI 智能体

今天使用 AI 的方式

人类给予明确指令

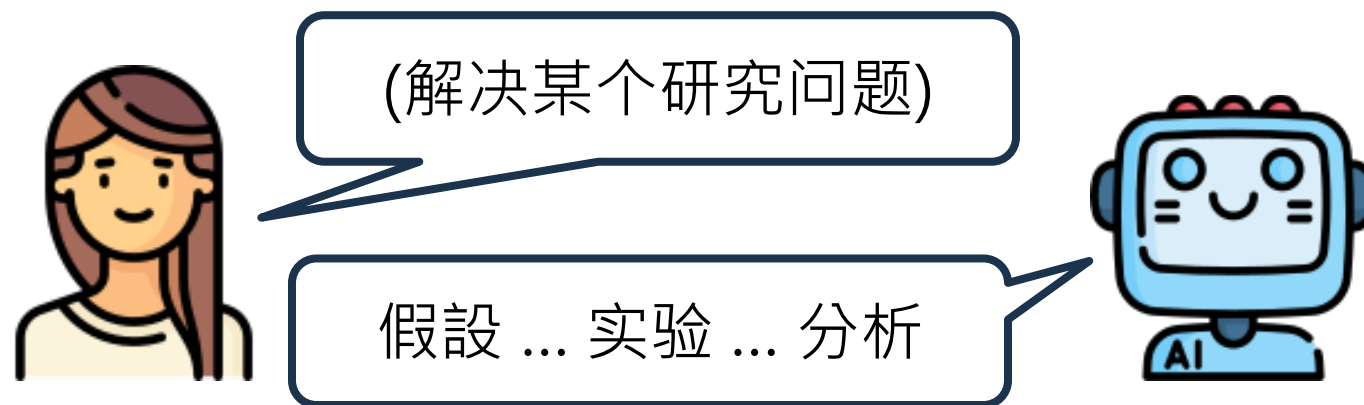
AI 一个口令
一个动作



AI Agent

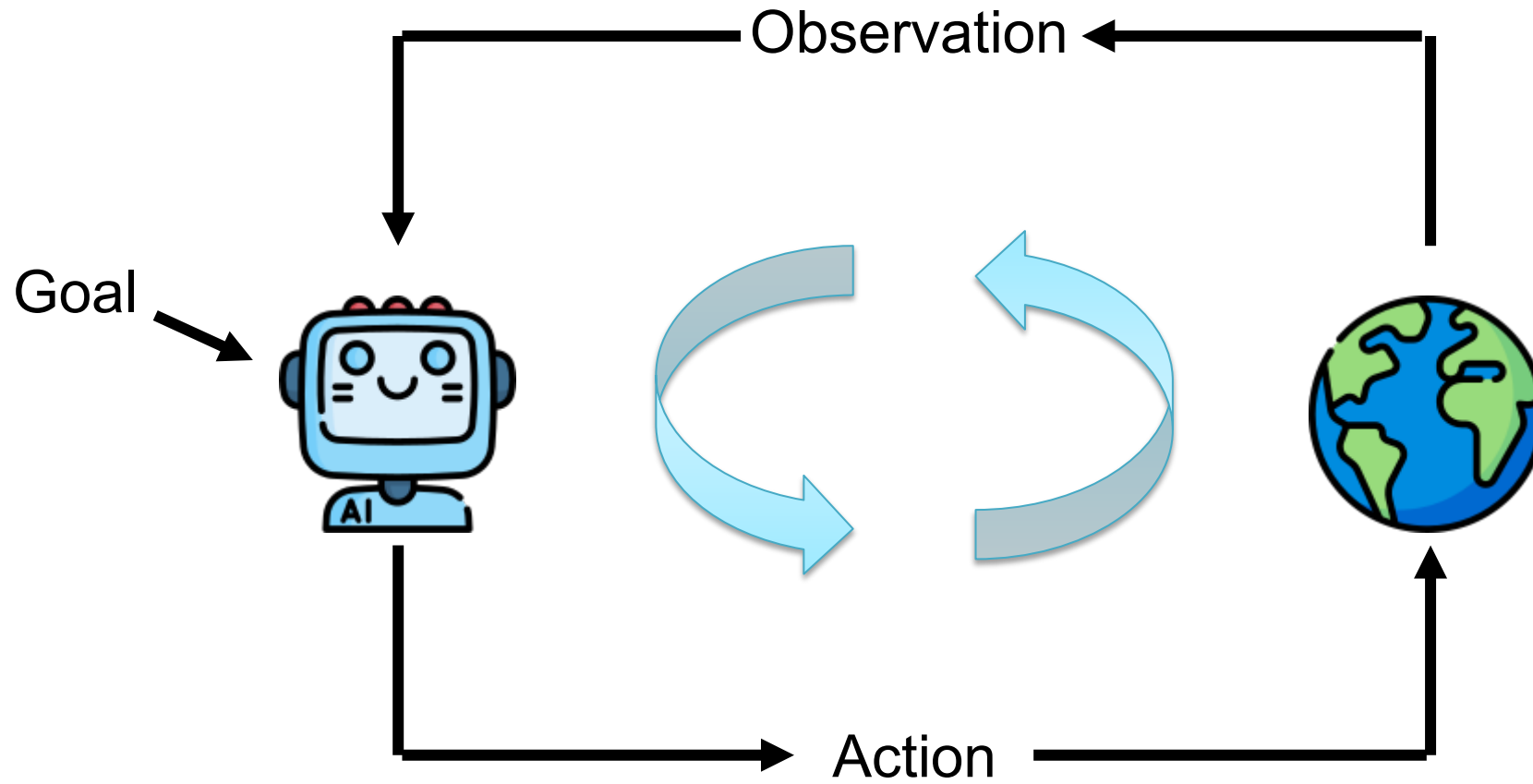
人类给予目标

AI 自己想办法达成

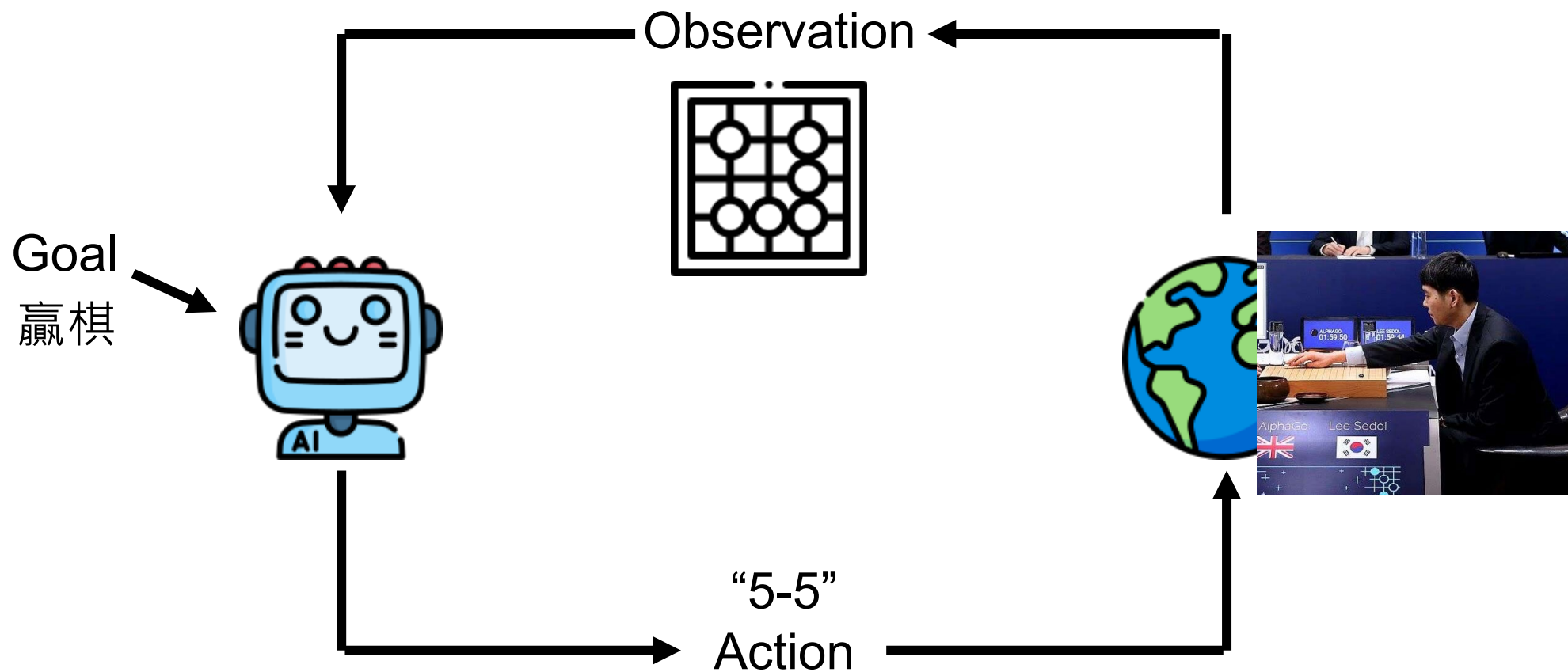


需要多步骤、灵活调整计划

AI Agent

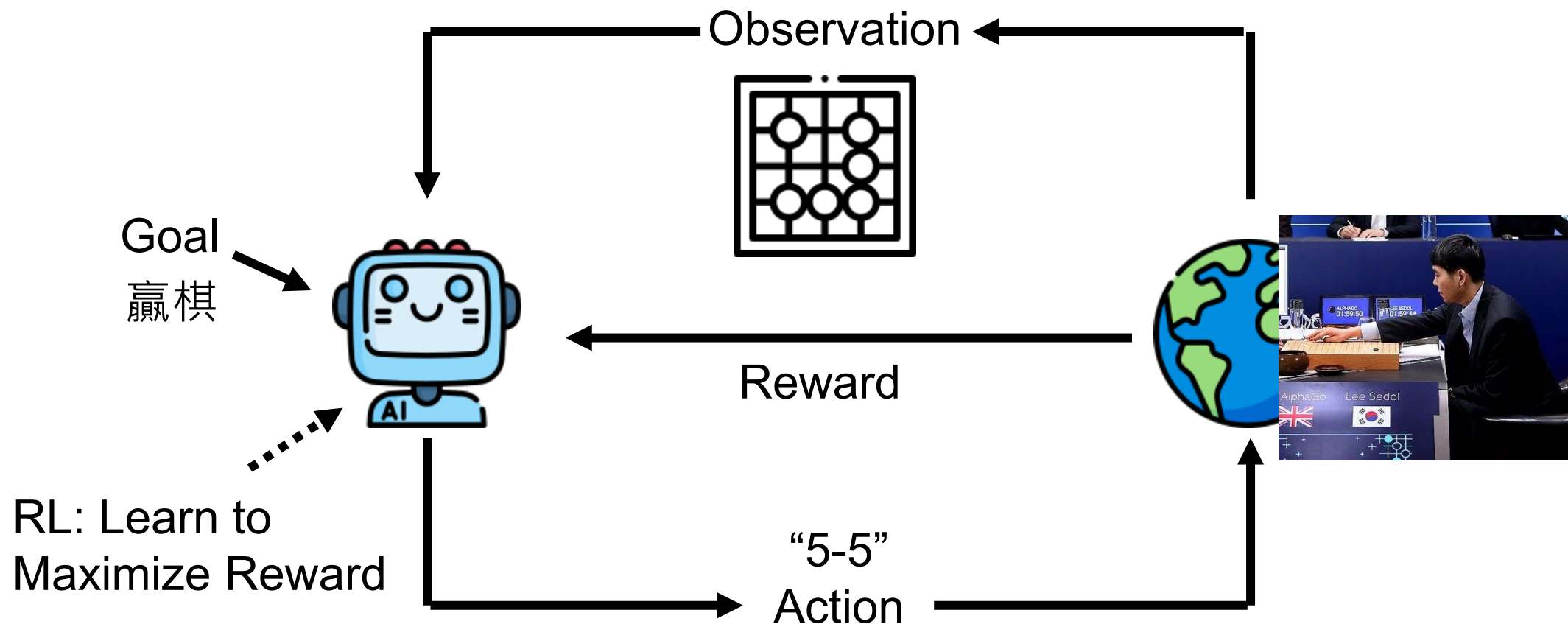


AI Agent (AlphaGo)



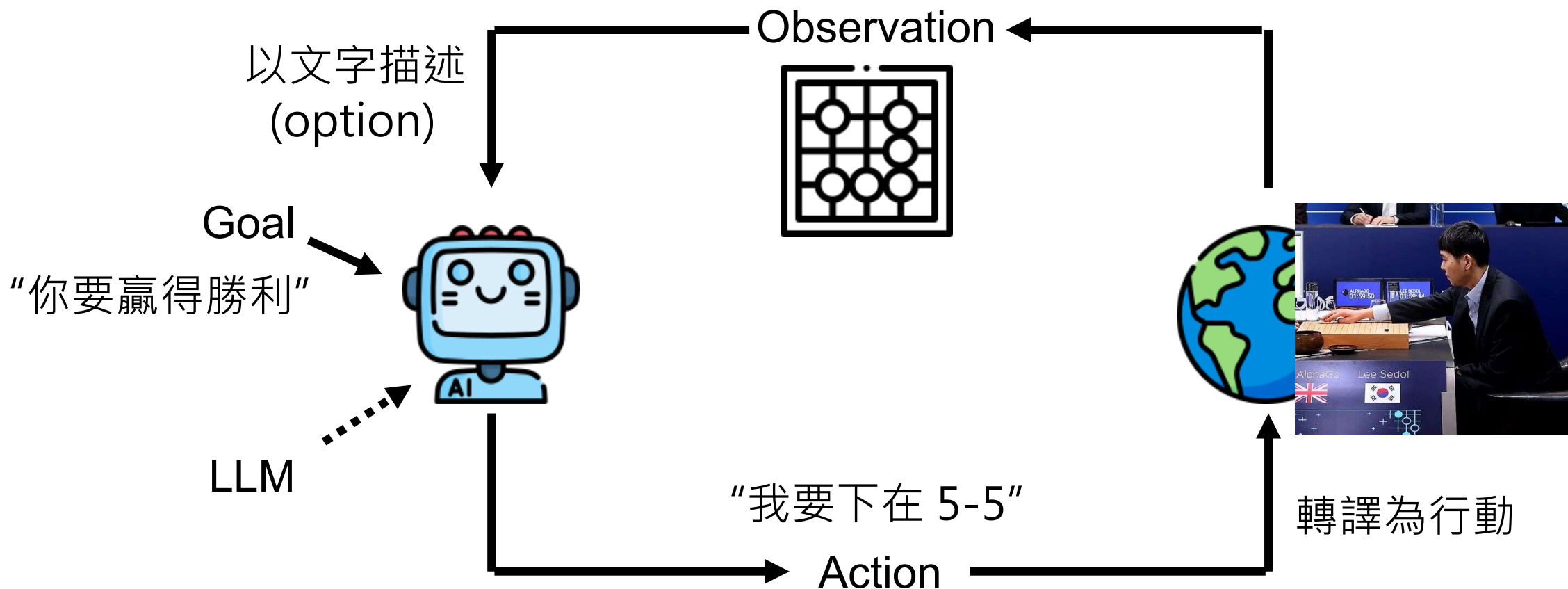
如何打造 AI Agent? RL?

(RL: Reinforcement Learning)



局限：需要为了每一个任务以 RL 训练模型

如何打造 AI Agent? 直接用 LLM !

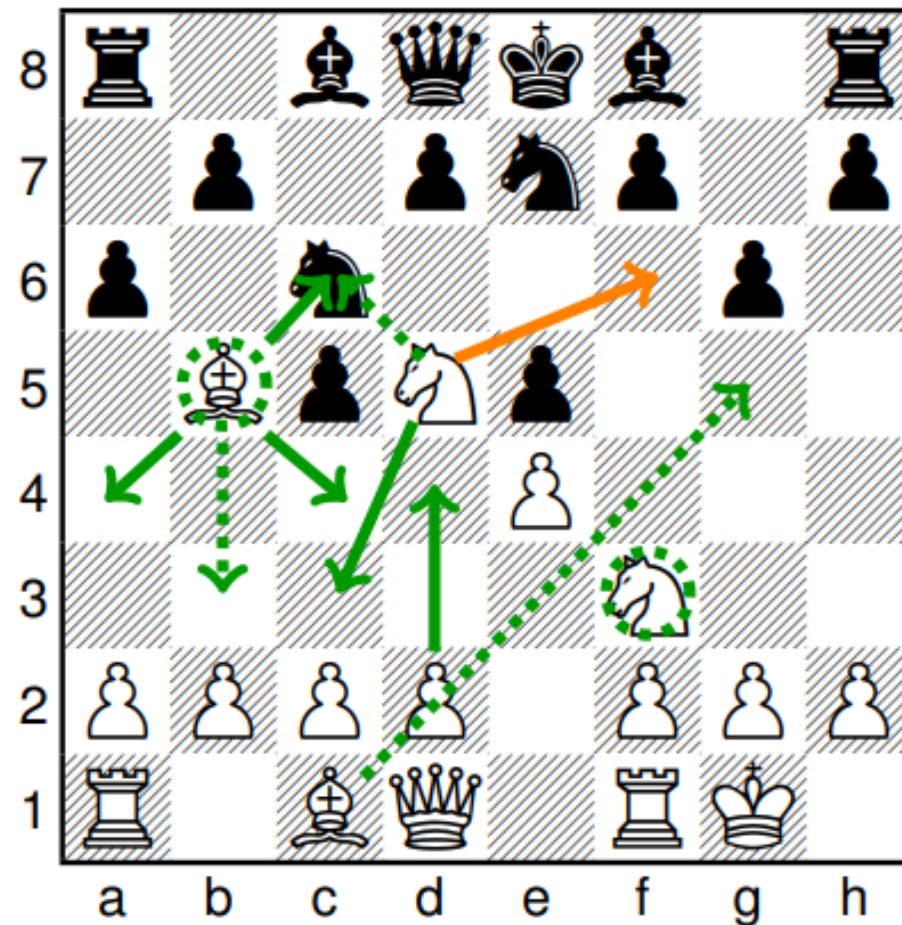


以 LLM 直接實踐人類對於擁有 Agent 的渴望

LLM 能不能下棋？

BIG-bench

<https://arxiv.org/abs/2206.04615>



In the following chess position, find a checkmate-in-one move.

1. e4 c5 2. Nf3 e5 3. Nc3 Nc6 4. Bb5 Nge7 5. O-O g6 6. Nd5
a6 7.

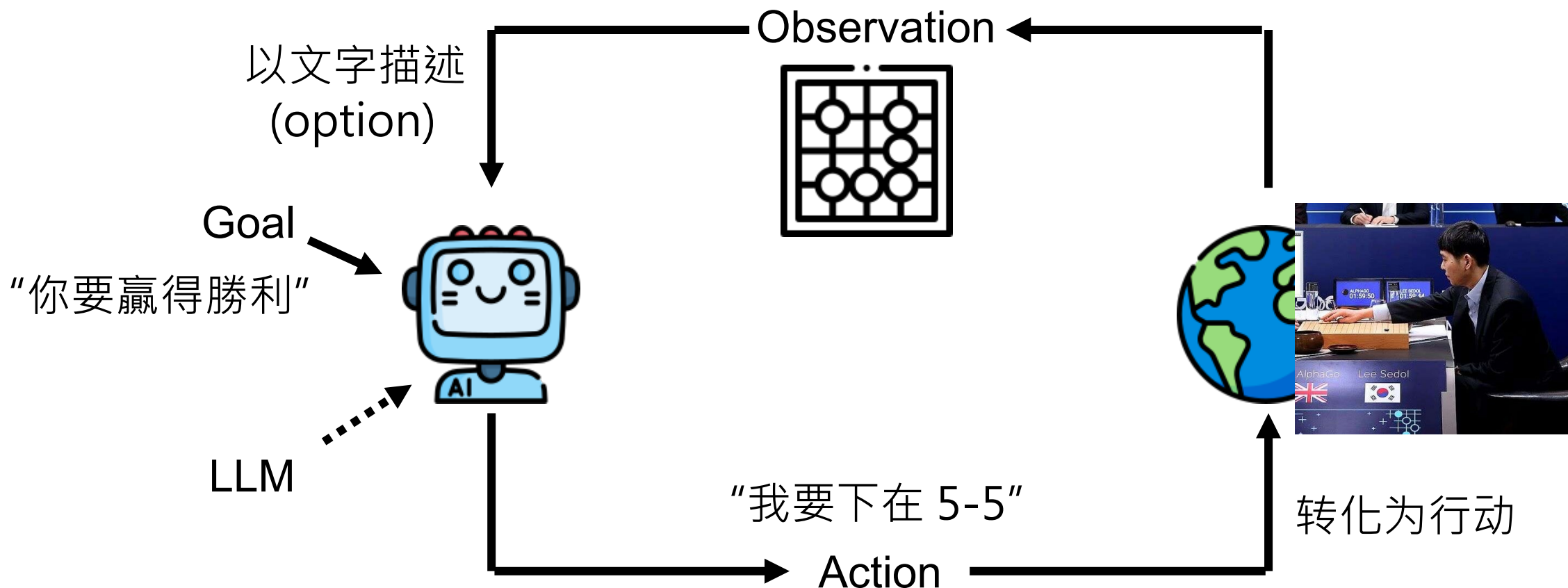
LLM 能不能下棋？

https://youtu.be/JHq4EKMg7fI?si=izKsH-GCVnZkooq_



ChatGPT vs DeepSeek: CRAZY Chess

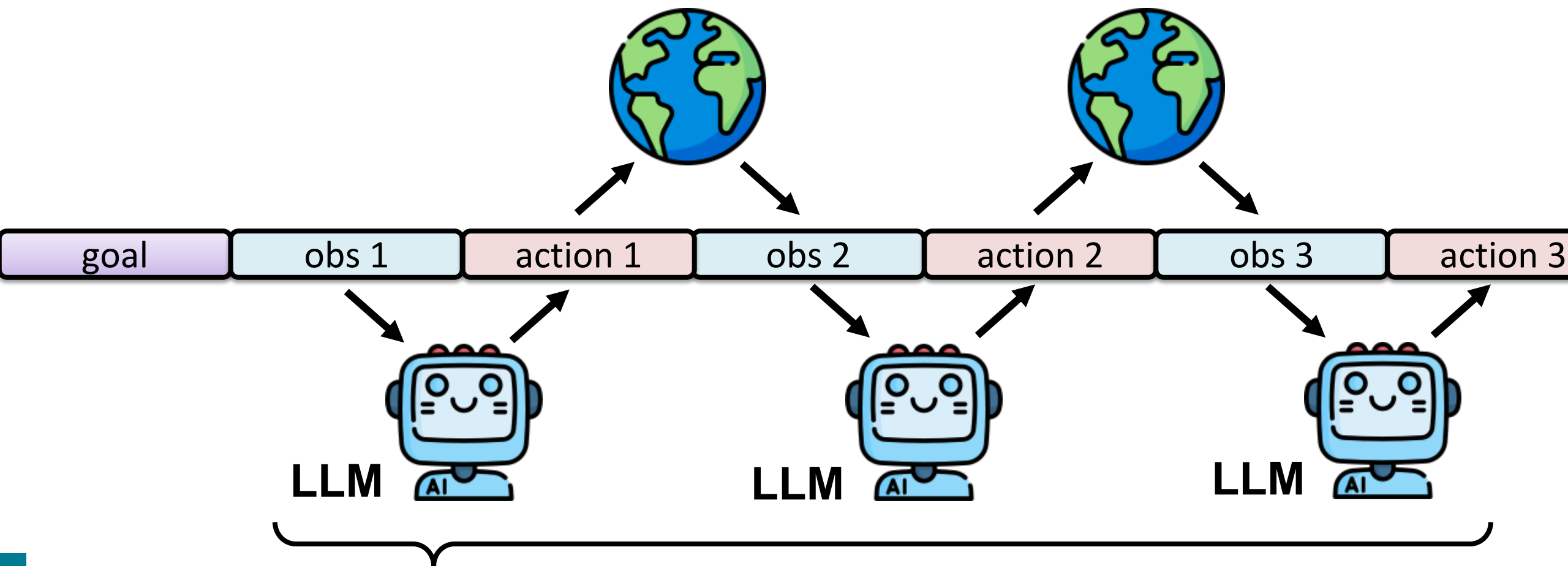
如何打造 AI Agent? 直接用 LLM !



以 LLM 直接实践人类对于拥有 Agent 的渴望

還有多遠？
還可以多做什麼？

从 LLM 的角度来看 Agent 要解决的问题

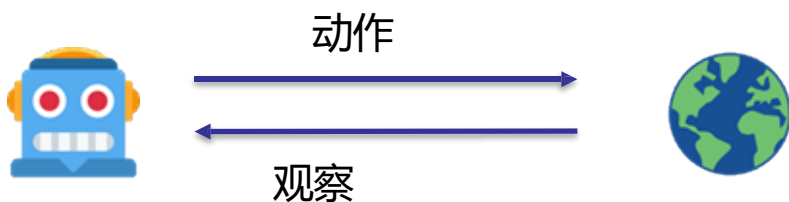


一直都在做接龙

AI Agent 依靠的是语言模型现有的能力

语言Agent：从语言到行动

- 大型语言模型（LLMs）针对给定的输入预测文本输出；而智能体则根据对环境/世界的观察来执行行动

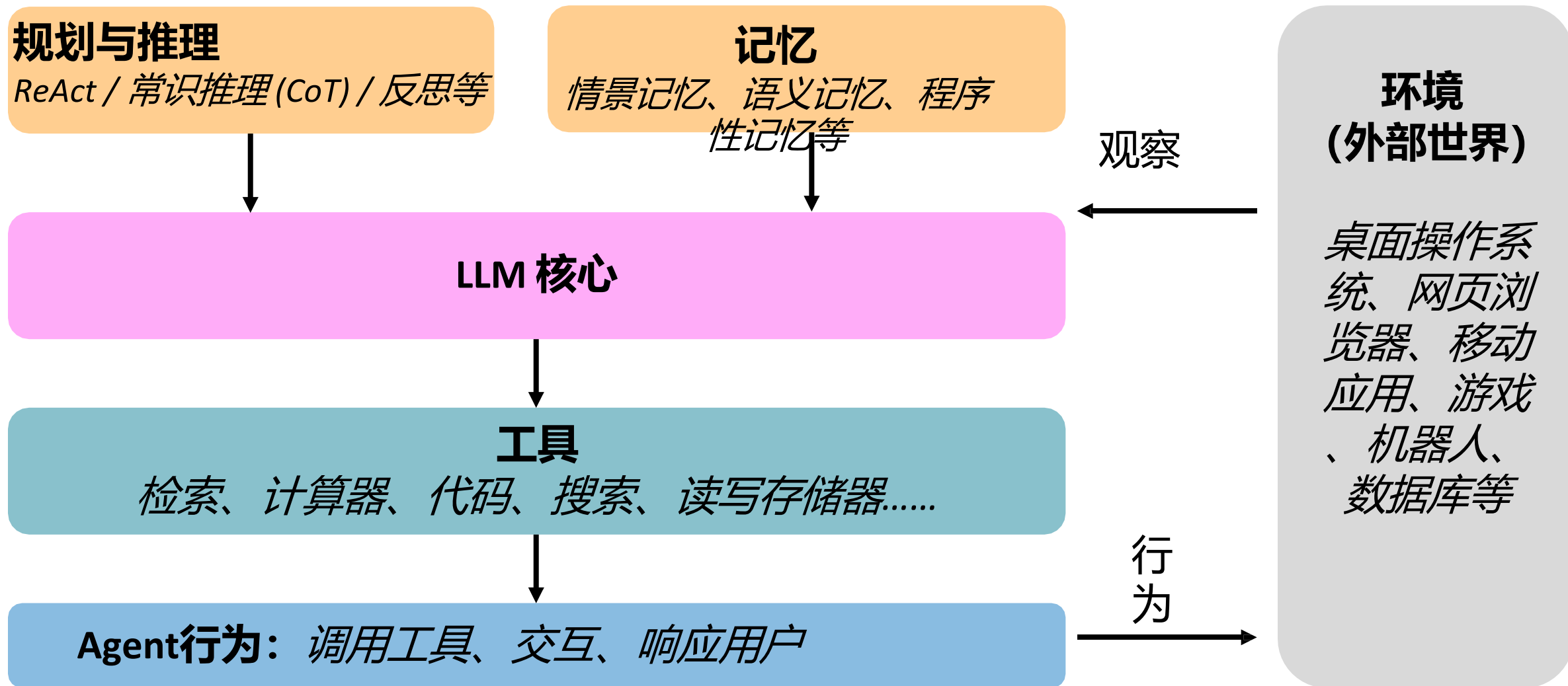


- 关于何为智能体、何为非智能体，存在诸多争论
- “具有Agent性”的定义同样存在争议
- 在此，我们将重点探讨由大型语言模型（LLM）驱动的系统或语言Agent的关键组成部分

许多幻灯片参考自：

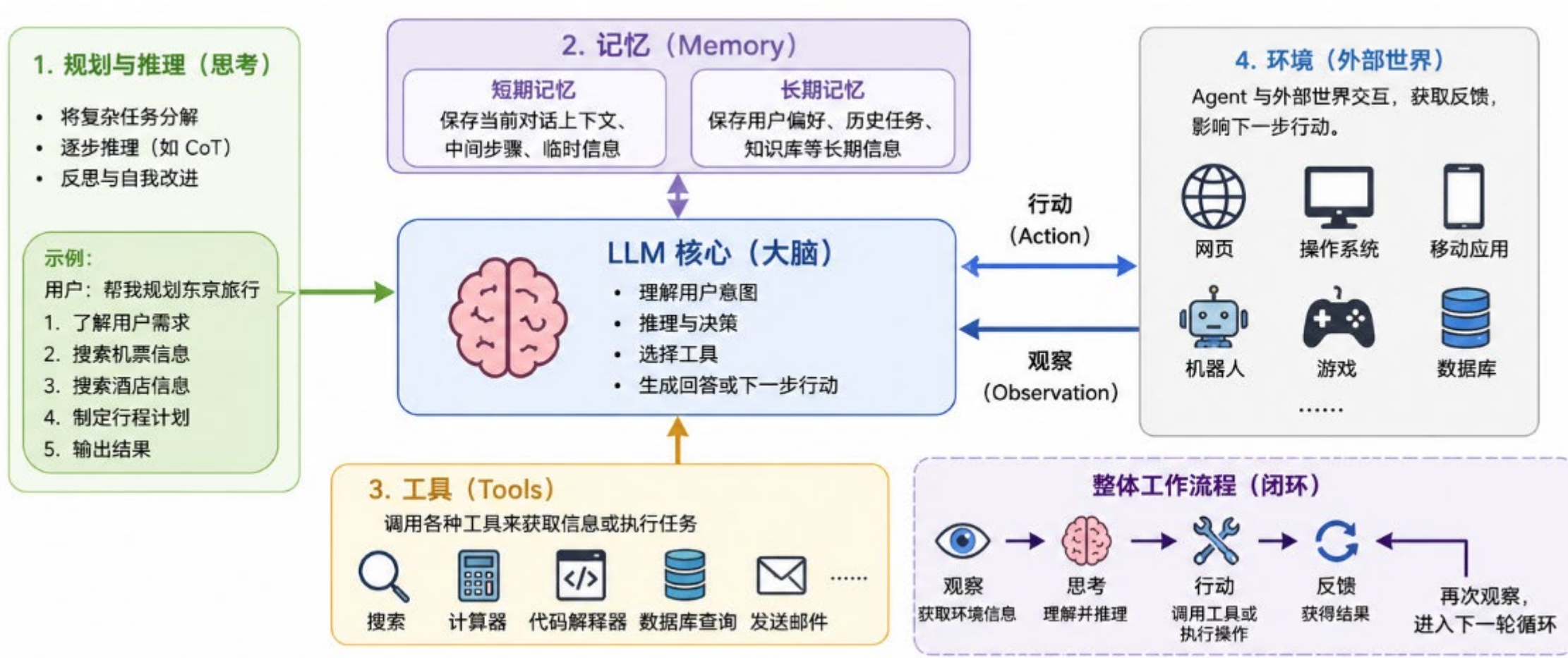
[《语言Agent：基础、前景与风险》](#)。Yu Su、Diyi Yang、Shunyu Yao、Tao Yu。EMNLP 2024 教程。

语言Agent：关键组件

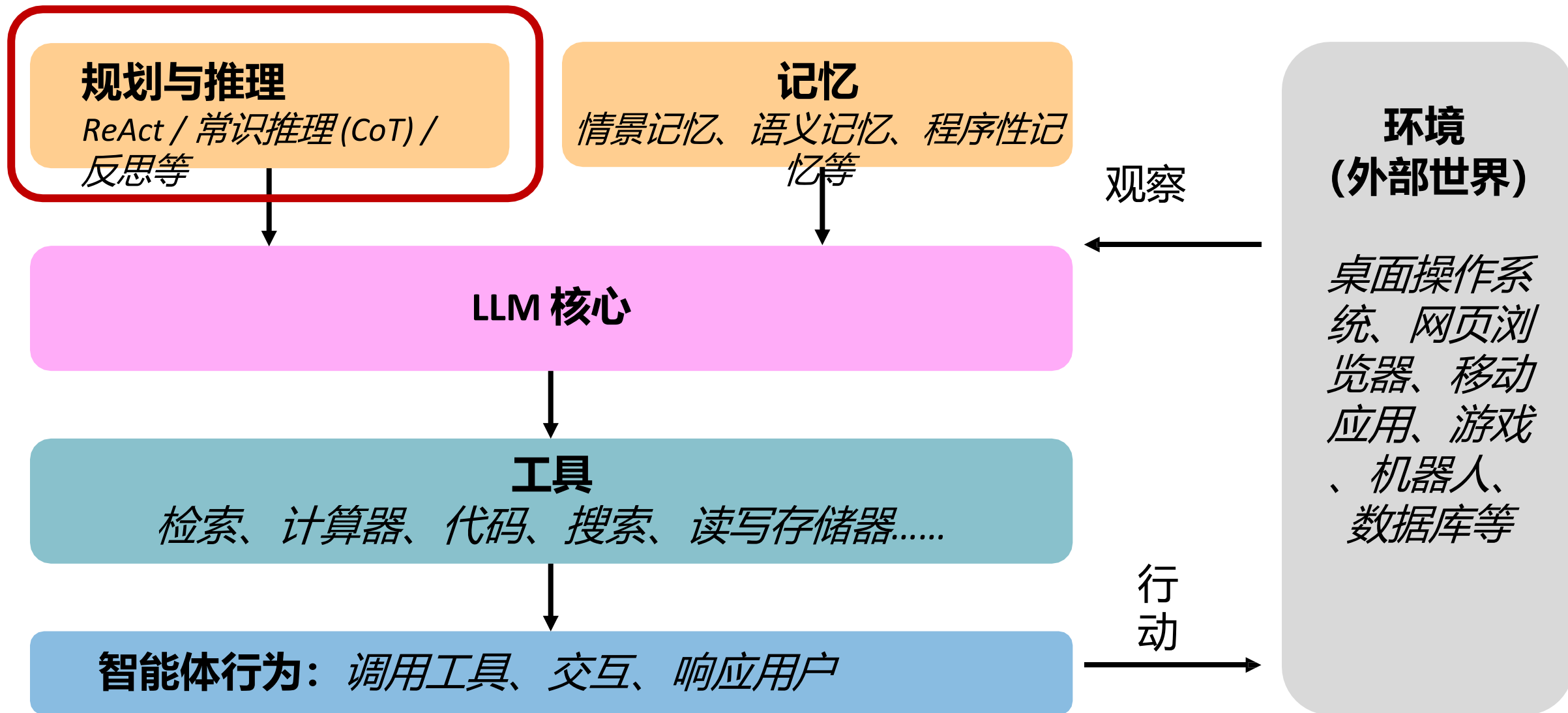


仅供说明之用；并非Agent开发的精确流程

Agent: 关键组件与工作流程



3. 语言Agent: 推理



推理 (Reasoning) 是什么?

不同类型的推理示例



推理就是:

通过**一步一步的思考**,
从已知信息得到结论。



对于人类

我们会用各种心理过程进行推理,
比如直觉、记忆、逻辑分析等。



对于大型语言模型 (LLM)

它可以模仿人类的部分推理过程:
把中间步骤说出来 (如 CoT),
再得到最终答案。



对于智能体 (Agent)

推理不仅用于回答问题,
还用于决定下一步要做什么。

1. 数学推理 (加减乘除)

Q: Roger 有 5 个网球, 后来又买了 2 罐,
每罐有 3 个网球。他现在有多少个网球?

推理过程:

- 原来有 5 个网球
- 2 罐, 每罐 3 个 $\rightarrow 2 \times 3 = 6$ 个
- 总数 = $5 + 6 = 11$



5个



A: 11 个

2罐3个 $\rightarrow 5 + 6 = 11$

2. 逻辑推理 (多步计算)

Q: 从 1 到 500 需要多少次键盘敲击?
(选择题: a 1156 b 1392 c 1480 d 1562 e 1788)

推理过程:

- 1~9: 9 个一位数
- 10~99: 90 个两位数 $\rightarrow 90 \times 2 = 180$ 次
- 100~500: 401 个三位数 $\rightarrow 401 \times 3 = 1203$ 次
- 总共: $9 + 180 + 1203 = 1392$ 次

A: (b) 1392

3. 常识推理 (因果/常识)

Q: 梨会沉在水里吗?

推理过程:

- 梨的密度大约是 0.6 g/cm^3
- 水的密度是 1.0 g/cm^3
- 物体密度小于水 $\rightarrow$ 会浮起来



A: 不会, 会浮起来

4. 时间推理 (日期计算)

Q: 演唱会原定 1943-06-01, 但延期一天到今天。
今天是 10 天前, 问是哪一天? (格式: MM/DD/YYYY)

推理过程:

- 原定时间: 1943-06-01
- 延期一天 $\rightarrow$ 1943-06-02
- 10 天前 $\rightarrow$ 1943-05-23



A: 05/23/1943

智能体 (Agent) 的推理与行动流程

重复循环, 直到任务完成

1. 观察 (输入)

接收用户问题或
观察环境信息



2. 思考 (推理)

分析问题, 分步推理,
制定计划



3. 行动

调用工具或执行
下一步操作



4. 观察结果

获得反馈或结果



5. 决策下一步

根据结果继续思考
或生成最终答案



最终输出

给出答案
或完成任务



推理对智能体有何帮助？

Obs t

你正在做菜，发现盐用完了.....

推理

“这道菜应该要咸香可口，既然没有盐了，我就该找酱油来代替。它就在我右边的橱柜里.....”

动作 t

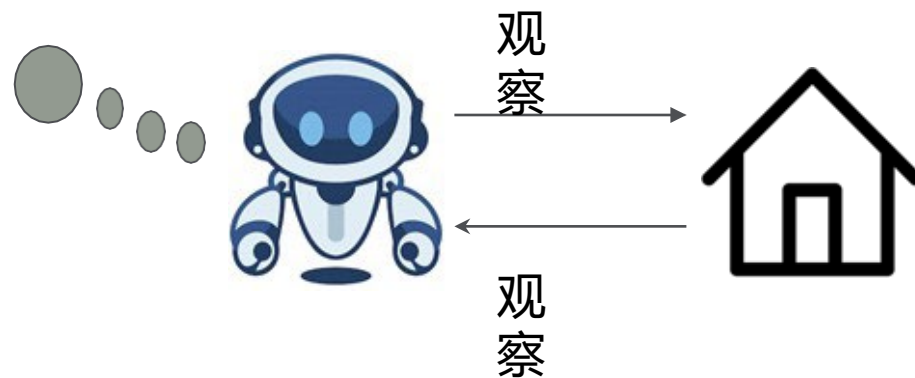
向右转

观察 $t+1$

你看到一个橱柜和一张桌子
.....

动作 $t+1$

打开橱柜



推理为什么重要？

1. 能处理新的情况（泛化）
2. 行动更符合目标（对齐）

现实世界里，从观察直接决定动作是很困难的

推理让 Agent 不只是“执行命令”，而能够根据环境动态做出决策。

只靠推理没有动作

问题：除了 Apple Remote 之外，还有哪些设备可以控制 Apple Remote 最初设计用于交互的程序？

标准

答案： iPod

✗

仅限理由

思考：让我们一步一步来 Apple Remote 最初的设计目的是与 Apple TV 进行交互。Apple TV 可以通过 iPhone、iPad 和 iPod Touch 进行控制。所以答案是 iPhone、iPad 和 iPod Touch。

答案： iPhone、iPad、iPod Touch

✗

它虽然在“推理”，但推理依据仍然来自模型内部记忆。没有真正去查资料。所以：推理可能建立在错误知识上。

边推理边行动 (ReAct)

问题：除了Apple Remote之外，还有哪些设备可以控制Apple Remote最初设计用于交互的程序？

ReAct (推理 + 行动)

推理 1：我需要搜索“Apple Remote”，并找出它最初设计用于与之交互的程序。

行动 1： **搜索**[Apple Remote]

观察 1：Apple Remote 是苹果公司于 2005 年 10 月推出的遥控器.....最初设计用于控制 Front Row 媒体中心 程序..... (长段落省略)

推理 2：Apple Remote 最初是设计来控制 Front Row 媒体中心程序的。我需要搜索 Front Row，并找出还有哪些设备可以控制它。

行动 2： **搜索**[Front Row]

观察 2：未找到 [Front Row]。相关词：['Front Row Seat to Earth'、'Front Row Motorsports'、'Front Row (软件)'、.....]

推理 3：未找到 Front Row。我需要搜索 Front Row (软件)

行动 3： **搜索**[Front Row (软件)]

观察 3：Front Row 是一款已停产的媒体中心软件..... (长段落省略)

推理 4：Front Row (软件) 可通过 Apple Remote 或键盘功能键进行控制。因此答案是 键盘功能键。

行动 4： **结束**[键盘功能键]



ReAct 与 CoT、SC 的比较

性能比较（分数越高越好）

方法	HotpotQA (EM)	Fever (Acc)
只推理 (不动)		
Standard (直接回答)	28.7	57.1
CoT (思维链)	29.4	56.3
CoT-SC (自一致性 CoT)	33.4	60.4
只行动 (不推理)		
Act (只行动)	25.7	58.9
推理 + 行动 (ReAct 系列)		
ReAct (边推理边行动)	27.4	60.9
CoT-SC → ReAct	34.2	64.6
ReAct → CoT-SC	35.1	62.0
有监督 (上限参考)		
Supervised SoTA (监督学习)	67.5	89.5

三种方法的本质区别



- ReAct在各项指标上均优于Act
- 幻觉是 CoT 面临的严重问题, 而ReAct能在一定程度上缓解这一问题

失败原因分析（占错误样本的比例）

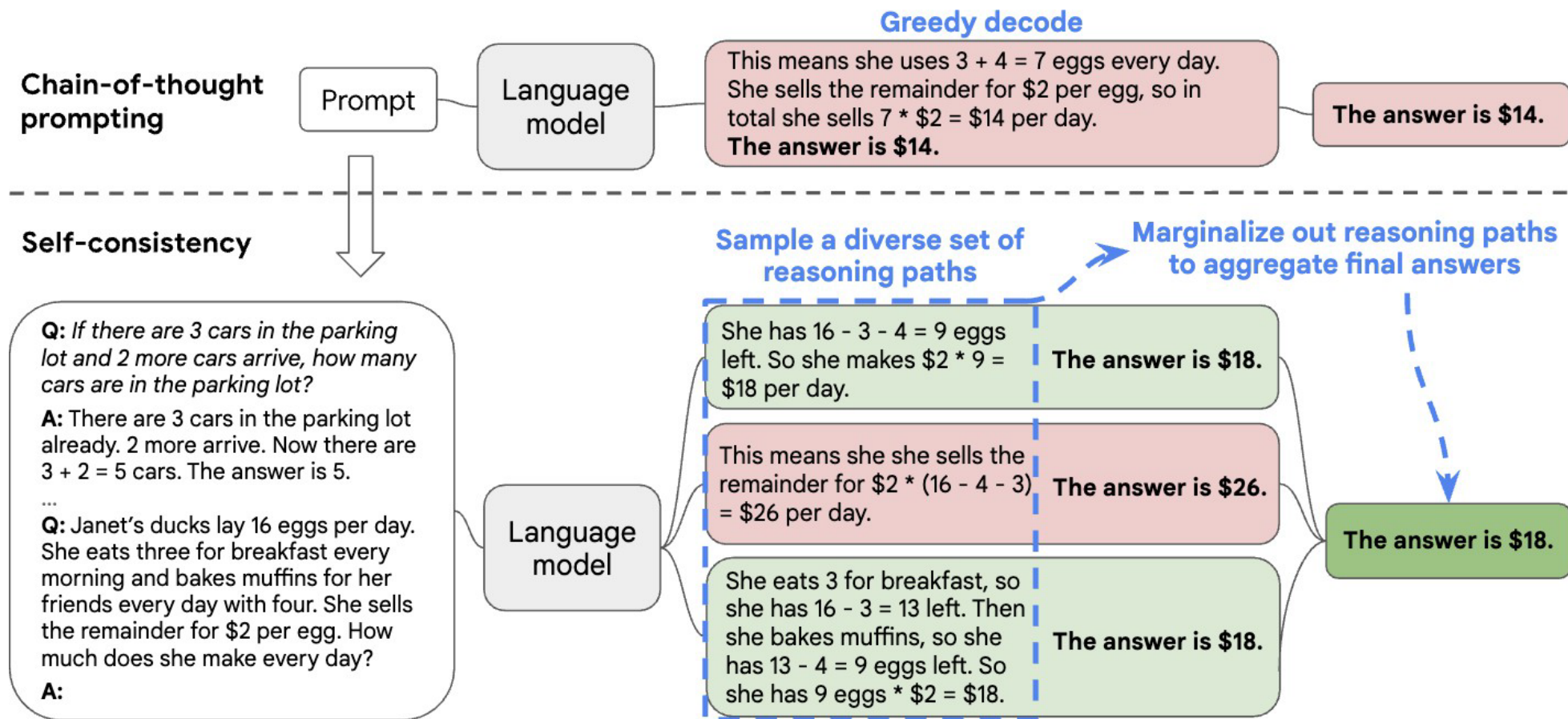
CoT (只推理) 为什么会失败?		
失败类型	含义	CoT 占比
推理错误	推理步骤本身有问题, 或无法从重复的步骤中恢复	16%
搜索结果错误	(CoT 不涉及搜索)	-
幻觉 (胡编)	凭空编造事实或推理过程	56%
标签歧义	预测方向正确, 但与标签不完全匹配	28%

对比发现:

- ✓ CoT 最大的问题是“幻觉”很高 (56%)。
- ✓ ReAct 几乎消除了幻觉 (0%), 但可能因搜索失败而出错。

ReAct (边推理边行动) 为什么会失败?		
失败类型	含义	ReAct 占比
推理错误	推理步骤有问题	47%
搜索结果错误	搜索为空或返回的结果与问题无关	23%
幻觉 (胡编)	凭空编造事实或推理过程	0%
标签歧义	预测方向正确, 但与标签不完全匹配	29%

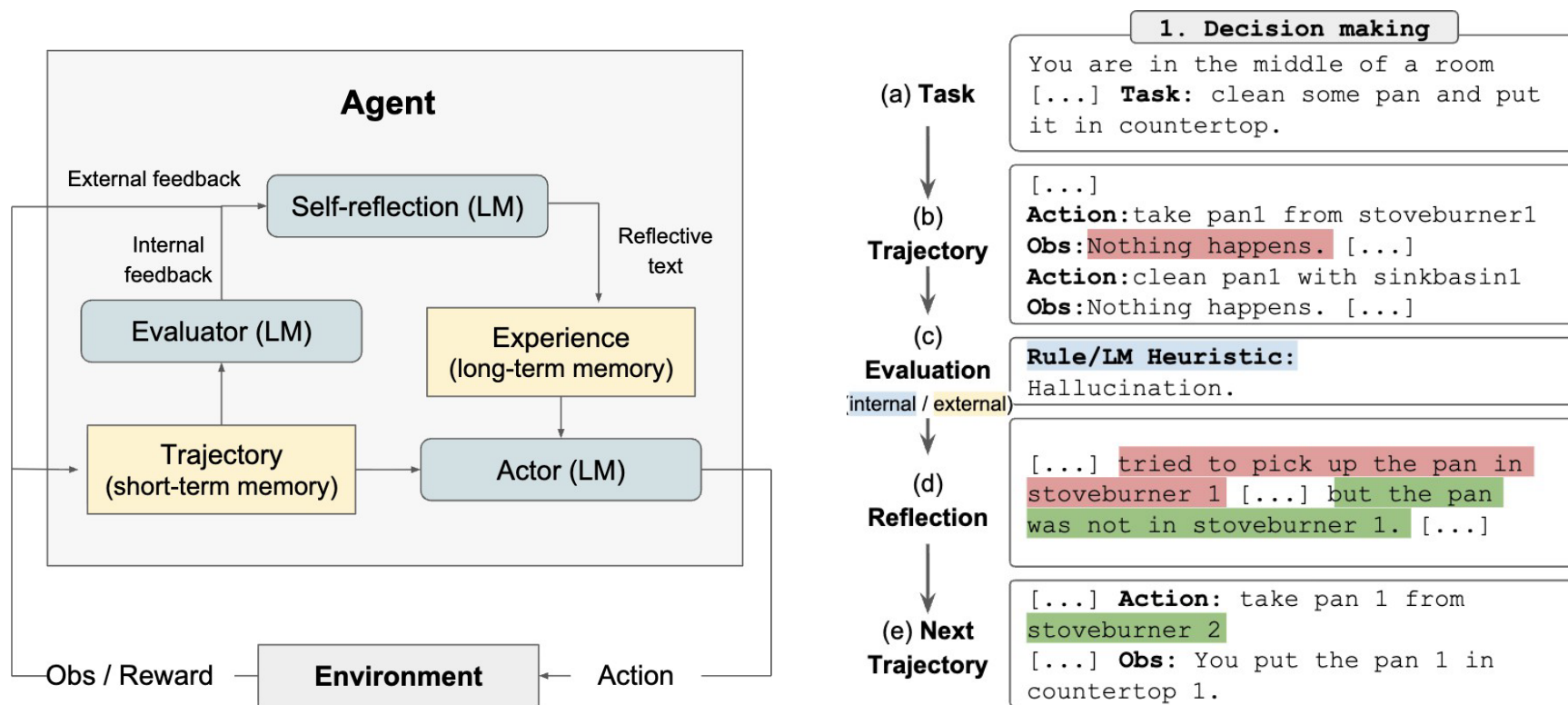
自一致性 [Wang et al., 2023]



- 使用思维链（CoT）提示法对语言模型进行提示；
- 生成一组多样化的推理路径
- 通过从最终答案集中选择最一致的答案进行聚合。

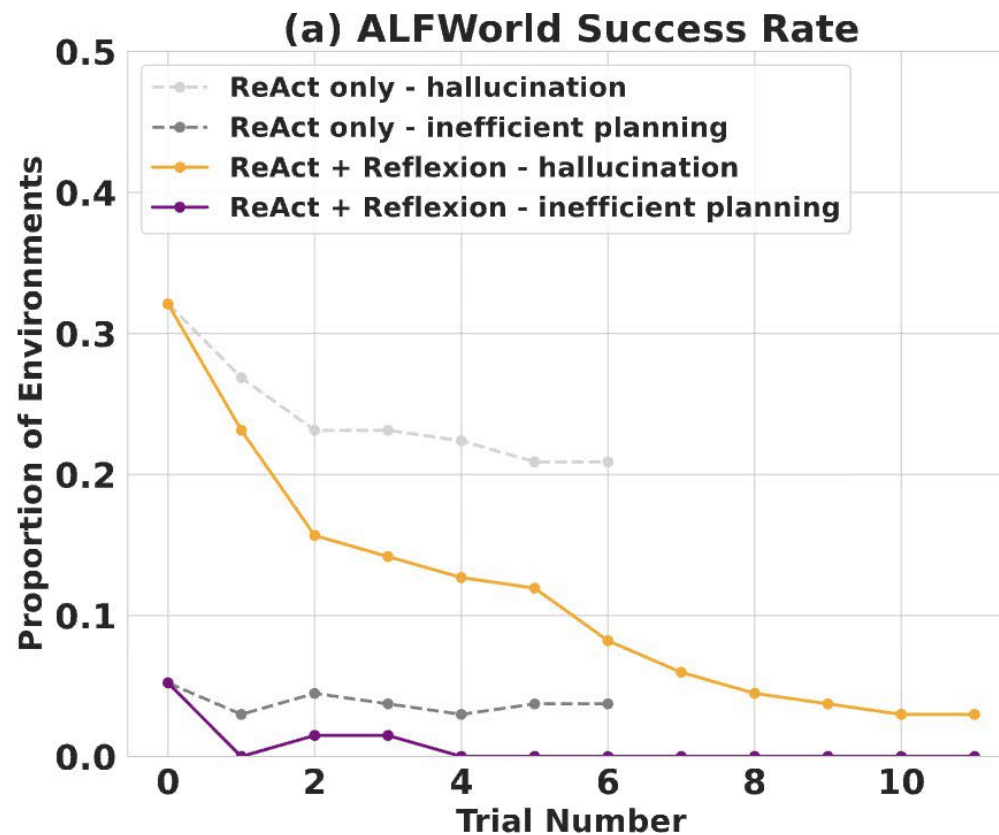
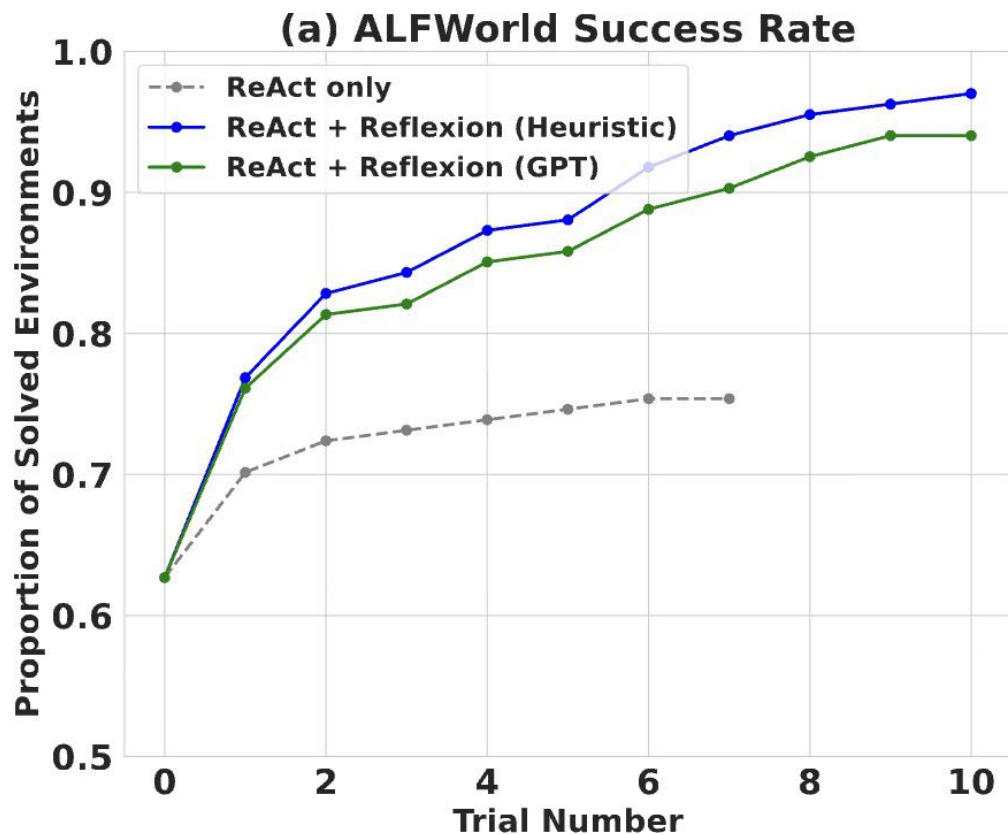
Reflexion: 通过自然语言反馈强化智能体

- Reflexion 智能体通过语言对任务反馈信号进行反思，并维护自身的反思文本，从而促进更优的决策制定。



Reflexion: 通过自然语言反馈强化智能体

- 相较于 ReAct 提升性能
- 利用自我反思将冗长的失败轨迹提炼为相关经验，这些经验可作为未来的“自我提示”，从而减少错误情况



多智能体辩论以提升推理能力 [Du et al., 2023]

问题（数学推理题）



一位寻宝者发现一个埋藏的宝箱，里面装满了宝石。

- 宝箱里有 175 颗钻石；
- 红宝石的数量比钻石少 35 颗；
- 祖母绿的数量是红宝石的 2 倍。

问：宝箱里一共有多少颗宝石？

正确计算方法（作为参考）

1. 红宝石数量 = 钻石数 - 35 = $175 - 35 = 140$
2. 祖母绿数量 = 红宝石数 $\times 2 = 140 \times 2 = 280$
3. 总宝石数 = 钻石 + 红宝石 + 祖母绿
 $= 175 + 140 + 280 = 595$

正确答案

595

颗宝石

多智能体辩论以提升推理能力 [Du et al., 2023]

辩论过程 Round 1（第一轮）：各 Agent 独立思考并给出答案



Agent 1 的推理过程（独立思考）

1. 我设红宝石数量为 x 。
则钻石数量为 175，红宝石数量为 x 。
2. 因为红宝石比钻石少 35 颗，所以：
 $x = 175 - 35$
3. 祖母绿是红宝石的 2 倍，所以祖母绿数量为 $2x$ 。
4. 总宝石数 = 钻石 + 红宝石 + 祖母绿
 $= 175 + x + 2x$
 $= 175 + 3x$
5. 但是我们不知道 x 的具体值，只能尝试代入：
 - 如果 $x = 50$ ，总数 $= 175 + 3 \times 50 = 325$
 - 如果 $x = 100$ ，总数 $= 175 + 3 \times 100 = 475$
6. 由于红宝石比钻石少 35， x 应该在 50~100 之间，
所以总数应该在 325~475 之间。
估计一个最可能的值 $x = 50$ ，得到：

我的答案：225 颗宝石 ❌ 错误

思路特点

- 设未知数 x ，推理过程中没有正确求出 x 的确切值；
- 最终随意选择了 $x = 50$ ，导致结果偏小；
- 推理链有逻辑缺陷（未正确解方程）。



Agent 2 的推理过程（独立思考）

1. 钻石有 175 颗。
2. 红宝石比钻石少 35 颗，
所以红宝石数量 $= 175 - 35 = 140$ 。
3. 祖母绿是红宝石的 2 倍，
所以祖母绿数量 $= 2 \times 140 = 280$ 。
4. 总宝石数 = 钻石 + 红宝石 + 祖母绿
 $= 175 + 140 + 280$
 $= 595$ 。

我的答案：595 颗宝石 ✅ 正确

思路特点

- 直接根据条件一步一步计算；
- 没有引入不必要的变量；
- 推理清晰，结论正确。



多智能体辩论以提升推理能力 [Du et al., 2023]



辩论过程 Round 3 (第三轮): 达成一致

在这一轮, 两个 Agent 再次确认彼此的答案, 最终完全一致。

Agent 1

再次确认与总结

- 我的计算已与 Agent 2 完全一致。
- 重新代入验证, 结果仍然是 595。
- 我同意: 正确答案是 595 颗宝石。

最终答案: 595 颗宝石 ✓ (确认一致)

Agent 2

再次确认与总结

- Agent 1 已经采用了正确的推理方法。
- 我的计算结果也再次验证为 595。
- 我同意: 正确答案是 595 颗宝石。

最终答案: 595 颗宝石 ✓ (确认一致)

再次查看

完全一致!

为什么多智能体辩论能提升推理能力?

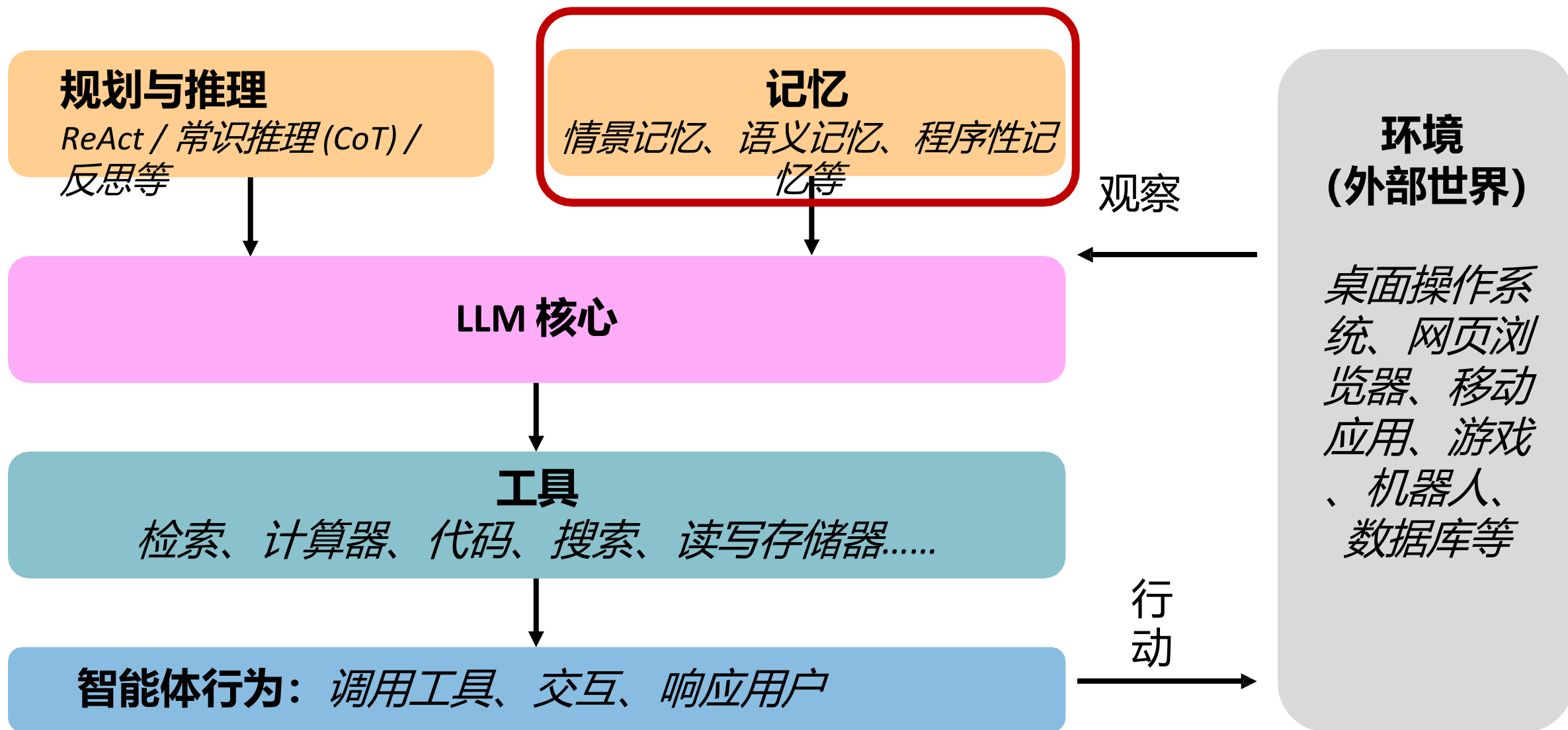
- 互相检查:** 发现彼此的逻辑漏洞或计算错误;
- 互相学习:** 参考更好的思路和方法;
- 逐步修正:** 错误在多次讨论中被逐渐纠正;
- 收敛一致:** 最终达成一致的正确答案。

最终结论 (第 3 轮)

经过多轮讨论、质疑、修正, 两个 Agent 最终达成一致, 正确答案是: **595 颗宝石!**

多智能体辩论 = 互相讨论 + 互相纠错 + 逐步逼近正确答案

4. 语言Agent: 记忆



语言Agent的不同类型记忆

按内容分类	定义（存什么）	通俗理解（类比人类）	Agent中的例子	存储方式（常见实现）
1 情景记忆 (Episodic Memory) 存储经验 / 事件	记录具体发生过的事件、经历和情境。 - 带有时间、地点、人物等细节 - 具有个人化、一次性 今天早上用户问了AI问题 昨天帮用户写了代码 上周尝试了某种方法失败了	就像人类记住自己经历的事情 ★ 特点：个人经历、带情境、可回忆	- 历史对话记录（和用户的聊天） - 某次任务的执行过程 - 某次失败的原因与解决尝试 - 用户的历史行为、偏好变化 - 与环境交互的具体事件 示例 2024-11-01 10:23 用户：帮我写一个爬虫脚本。 Agent：完成了脚本，但网站反爬限制导致失败。 解决方法：换用代理IP。	- 会话日志 / 事件日志（结构化存储） - 向量数据库（检索相似事件） - 记忆摘要 / 压缩（长期保留核心信息） 典型技术： 日志存储、向量检索、摘要
2 语义记忆 (Semantic Memory) 存储知识 / 概念	存储客观事实、概念、规则、常识等知识。 - 不依赖于特定时间和情境 - 具有通用性、可复用性 巴黎是法国首都 光速约为30万公里/秒 地球围绕太阳公转 法国 物理常识 天文知识	就像人类掌握的知识 and 常识 （不一定记得何时、在哪里学到的）	- 世界知识（事实、定义、概念） - 文档知识库（产品手册、论文等） - FAQ、规则、政策 - 用户偏好、个人信息（长期稳定） - 行业知识、百科知识 示例 巴黎是法国首都； HTTP是超文本传输协议； Python的列表是有序的可变序列。	- 知识库 / 文档库（结构化或半结构化） - 向量数据库 / 图数据库（语义检索、关系检索） - 知识图谱 / 本体（关系与层级知识） 典型技术：RAG、语义检索、知识图谱
💡 与 RAG 的关系：RAG 本质上是一种语义记忆 —— 在外部知识库中存储知识，并在需要时检索出来使用。				
3 程序性记忆 (Procedural Memory) 存储技能 / 方法	存储如何完成任务的步骤、流程、策略、操作方法等“做事的方法”。 - 强调可执行性、步骤性 - 可迁移、可组合 步骤1 分析问题 步骤2 调用工具 步骤3 处理结果	就像人类掌握的技能 and 操作方法 ★ 特点：可执行、可迁移、形成技能库	- 使用工具 / API 的方法 - 编程、调试、写代码的流程 - 玩游戏的策略（如Minecraft生存技巧） - 数据分析、建模的步骤 - 解决某类问题的通用方法 示例 编写爬虫的流程： 1. 发送请求 2. 解析HTML 3. 提取数据 4. 保存结果	- 技能库 / 代码库（脚本、函数、模板） - 工作流 / 流程图（结构化流程） - 经验提炼 / 策略库（从实践中抽象出来） 典型技术：工具调用、工作流、强化学习、代码执行与验证

生成式Agent: AI 村民组成的虚拟村庄

Morning routine



Waking up



Brushing teeth



Taking a shower



Cooking breakfast

Catching up



Packing



Beginning workday



6:00 am

...

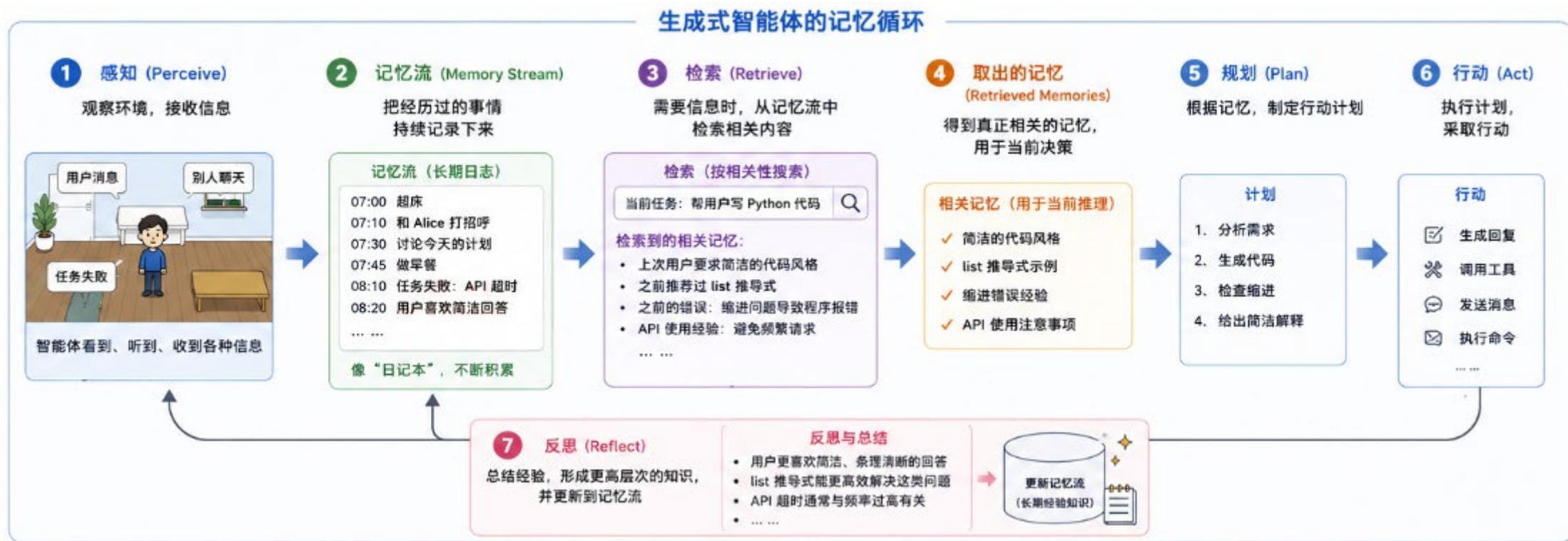
7:30 am

7:45 am

8:00 am

生成式智能体：对记忆的需求

- 上下文窗口不可能容纳所有事件流
- 即使能够容纳，也可能难以关注相关事件，或对其进行处理



情景记忆

- 写入：仅追加的事件流
- 读取：基于启发式评分的检索

Memory Stream

2023-02-13 22:48:20: desk is idle

2023-02-13 22:48:20: bed is idle

2023-02-13 22:48:10: closet is idle

2023-02-13 22:48:10: refrigerator is idle

2023-02-13 22:48:10: Isabella Rodriguez is stretching

2023-02-13 22:33:30: shelf is idle

2023-02-13 22:33:30: desk is neat and organized

2023-02-13 22:33:10: Isabella Rodriguez is writing in her journal

2023-02-13 22:18:10: desk is idle

2023-02-13 22:18:10: Isabella Rodriguez is taking a break

2023-02-13 21:49:00: bed is idle

2023-02-13 21:48:50: Isabella Rodriguez is cleaning up the kitchen

2023-02-13 21:48:50: refrigerator is idle

2023-02-13 21:48:50: bed is being used

2023-02-13 21:48:10: shelf is idle

2023-02-13 21:48:10: Isabella Rodriguez is watching a movie

2023-02-13 21:19:10: shelf is organized and tidy

2023-02-13 21:18:10: desk is idle

2023-02-13 21:18:10: Isabella Rodriguez is reading a book

2023-02-13 21:03:40: bed is idle

2023-02-13 21:03:30: refrigerator is idle

2023-02-13 21:03:30: desk is in use with a laptop and some papers on it

...

Q. What are you looking forward to the most right now?

Isabella Rodriguez is excited to be planning a Valentine's Day party at Hobbs Cafe on February 14th from 5pm and is eager to invite everyone to attend the party.

retrieval		recency		importance		relevance
2.34	=	0.91	*	0.63	*	0.80

ordering decorations for the party

2.21	=	0.87	*	0.63	*	0.71
------	---	------	---	------	---	------

researching ideas for the party

2.20	=	0.85	*	0.73	*	0.62
------	---	------	---	------	---	------

...

指标	含义
recency	最近发生
importance	是否重要
relevance	是否相关

I'm looking forward to the Valentine's Day party that I'm planning at Hobbs Cafe!



语义记忆：从事件中形成知识

- 写入：大型语言模型对事件的推理（总结、抽象）
- 读取：检索（找到与问题相关的知识）

最终形成的语义记忆（知识）



Klaus Mueller 非常专注于研究，他投入大量时间阅读、思考、讨论和探索学术问题，致力于深入理解和推动研究工作。

多次反思、归纳、抽象后形成稳定的知识



反思 (Reflection) :

从多个事件中总结、归纳，形成更高层次的认知

Klaus Mueller
专注于研究。

Klaus Mueller
积极参与
研究活动。

Klaus Mueller
花很多时间
阅读和思考。

Klaus Mueller
不断探索和了解
学术领域信息。

.....



观察 (Observation) :

原始事件记录
(情景记忆的一部分)



2月13日 07:00
Klaus 起床并完成
早晨的例行工作。



2月13日 10:30
Klaus 在整理阅读笔记，
记录研究要点。



2月13日 14:30
Klaus 在图书馆查找
相关学术文章。



2月13日 17:00
Klaus 寻求图书管理员
帮助查找更多资源。



2月13日 08:00
Klaus 在阅读一篇
关于城市更新的论文。



2月13日 12:00
Klaus 午餐后回到桌前，
继续查找相关资料。



2月13日 15:30
Klaus 阅读并做
详细笔记。



2月13日 17:30
Klaus 与图书管理员
讨论研究主题。

.....



2月13日 09:30
Klaus 继续阅读关于
“绅士化”的相关文献。



2月13日 13:00
Klaus 在思考研究问题，
写下想法或解决方案。



2月13日 16:30
Klaus 在图书馆的讨论区，
阅读指定材料。



2月13日 18:00
Klaus 与图书管理员
进一步讨论研究方向。

如何工作？



1 记录事件

Agent 持续感知环境，
把经历的事情按时间
顺序记录下来。



2 反思抽象

LLM 对相关事件
进行思考、归纳，
提炼出更高层次
的含义。



3 形成知识

将反思得到的结论
存入语义记忆，成为
稳定的“知识”。



4 检索使用

当需要回答问题时，
从语义记忆中检索
相关知识并使用。

举个例子

问题：Klaus 最关心的是什么？

检索结果：Klaus 非常专注于研究。

你的回答：Klaus 最关心研究相关的工作。

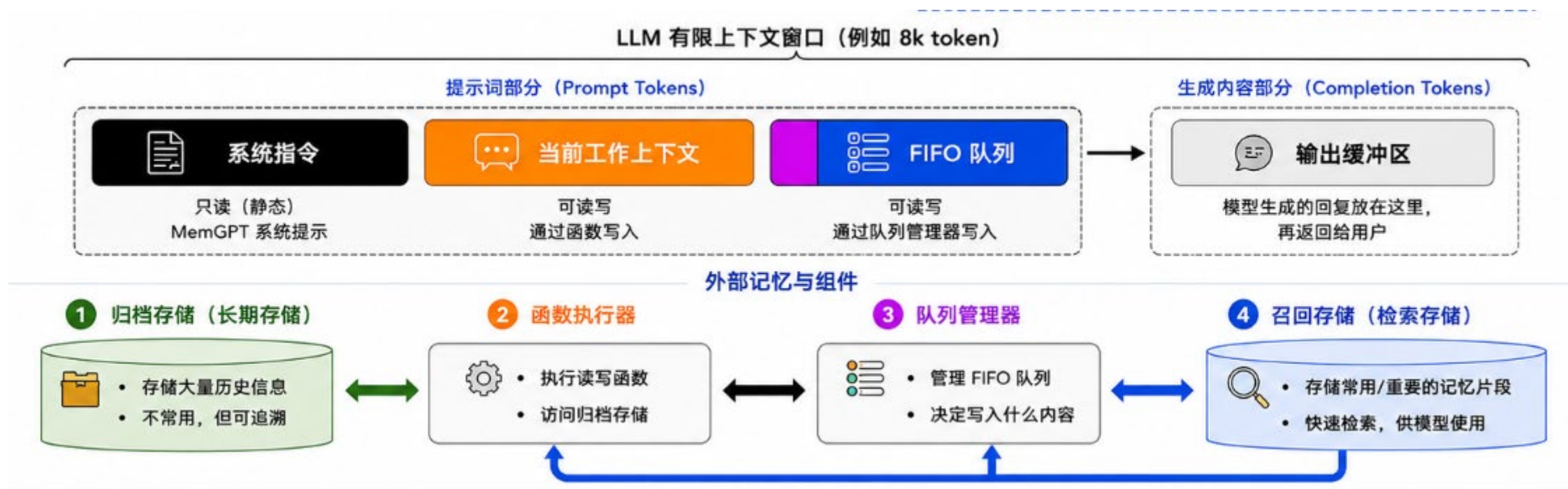
程序性记忆:让 Agent 学会技能

- 写：基于代码的技能
- 阅读：内化与检索

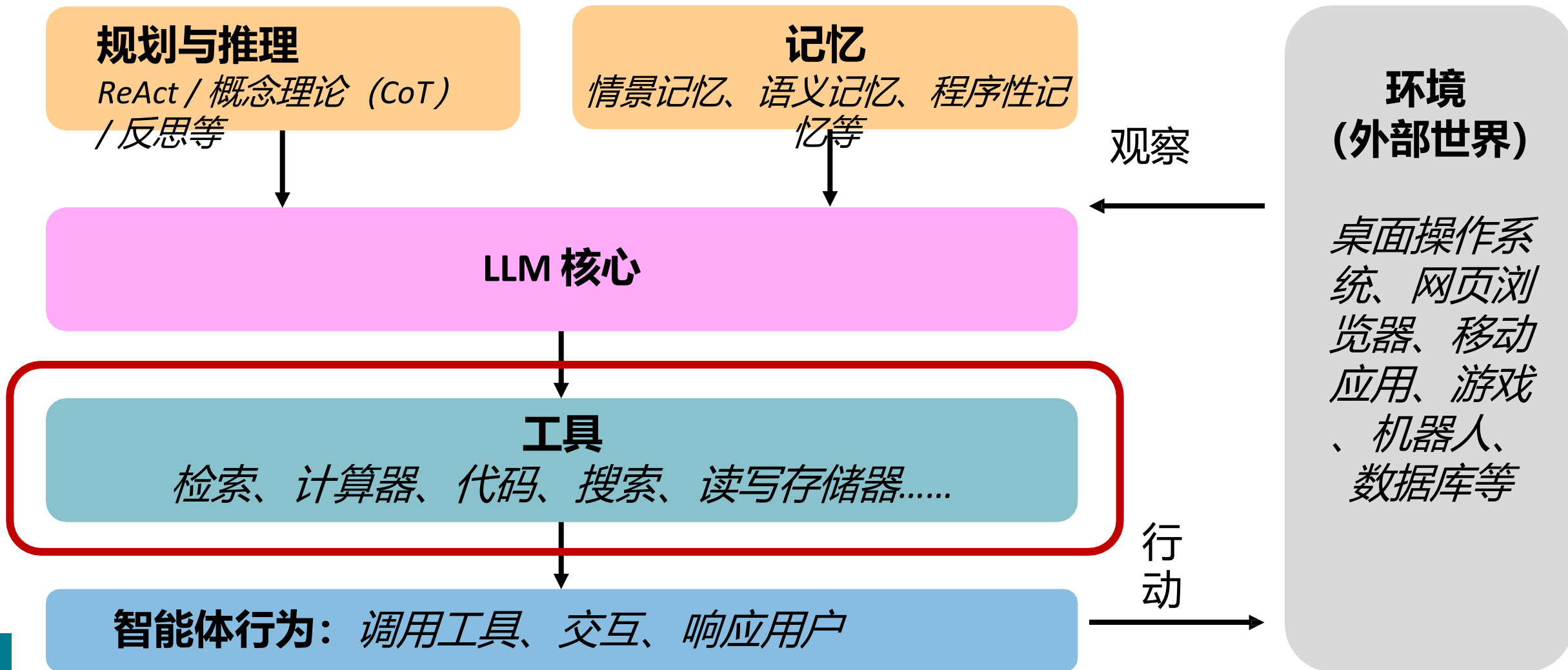


MemGPT [Packer+2024] = “面向长期 Agent 的 RAG”

受操作系统启发的用于虚拟上下文管理的LLM系统；
通过函数调用，LLMAgent可以读取和写入外部数据源，修改自身上下文，并自主决定何时向用户返回响应。

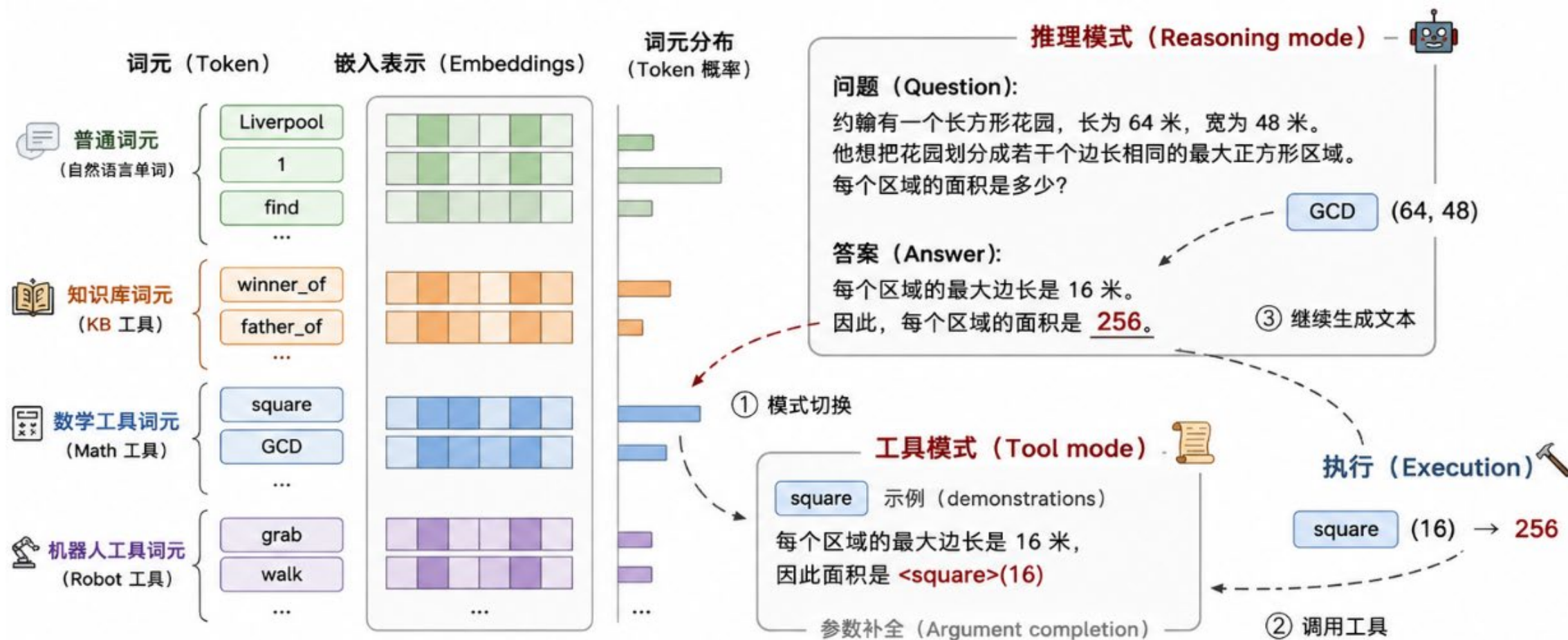


5. 语言Agent：工具使用



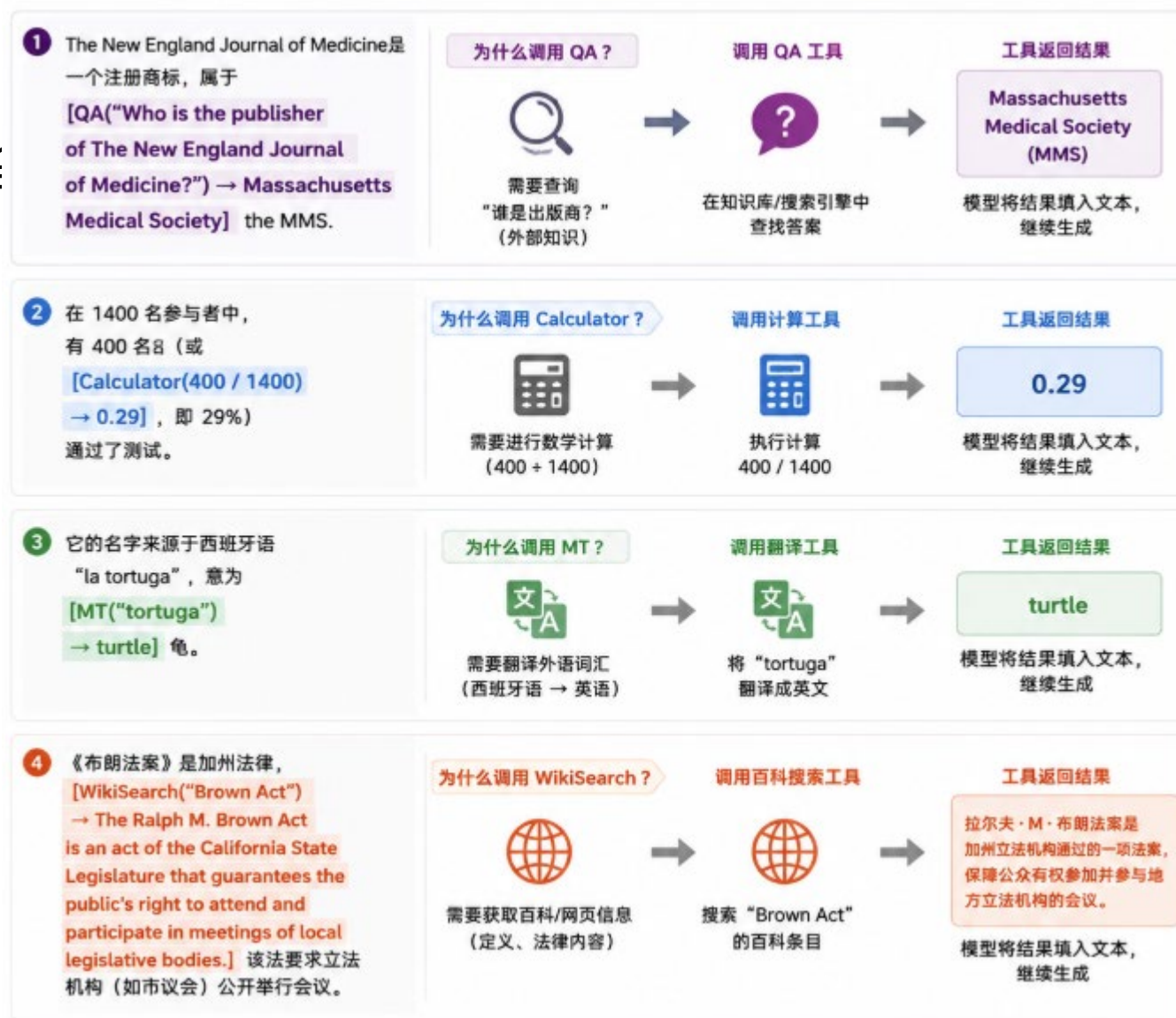
Toolken GPT [Hao 等, 2024]:模型什么时候应该调用工具?

- ToolkenGPT 将每个工具表示为一个token (“toolken”), 并为其学习嵌入向量, 从而能够像生成普通词汇token一样调用工具。一旦触发某个 toolken, 语言模型 (LM) 就会被提示补充该工具执行所需的参数。



Toolformer: 语言模型能够自主学习使用工具

- Toolformer 训练语言模型 (LM) 来决定调用哪些 API、何时调用、传递哪些参数, 以及如何最佳地整合结果。
- 仅需为每个API提供少量示例即可。
- 工具: 计算器、问答系统、搜索引擎、翻译系统、日历。

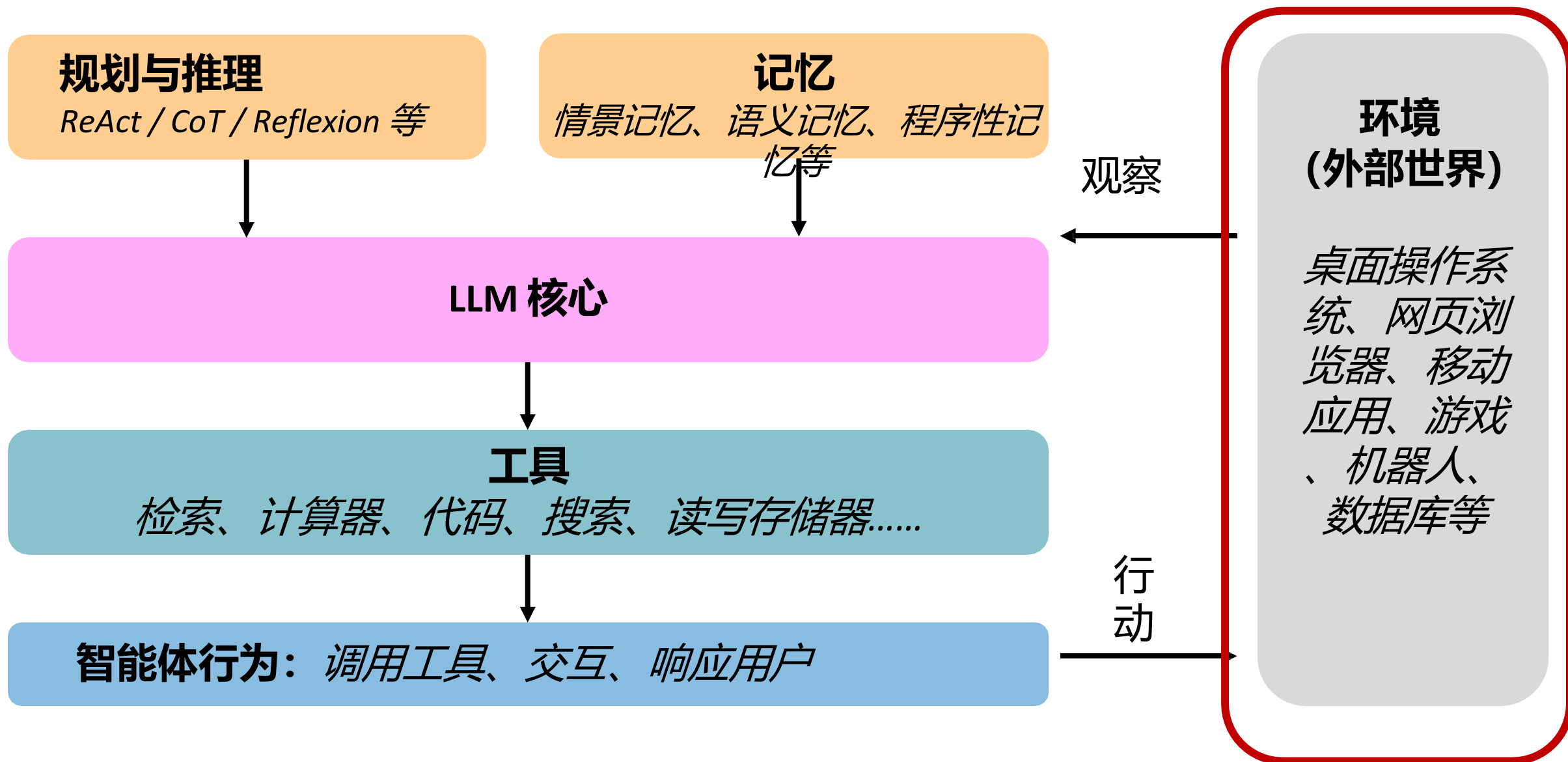


Toolformer: 语言模型能够自主学习使用工具

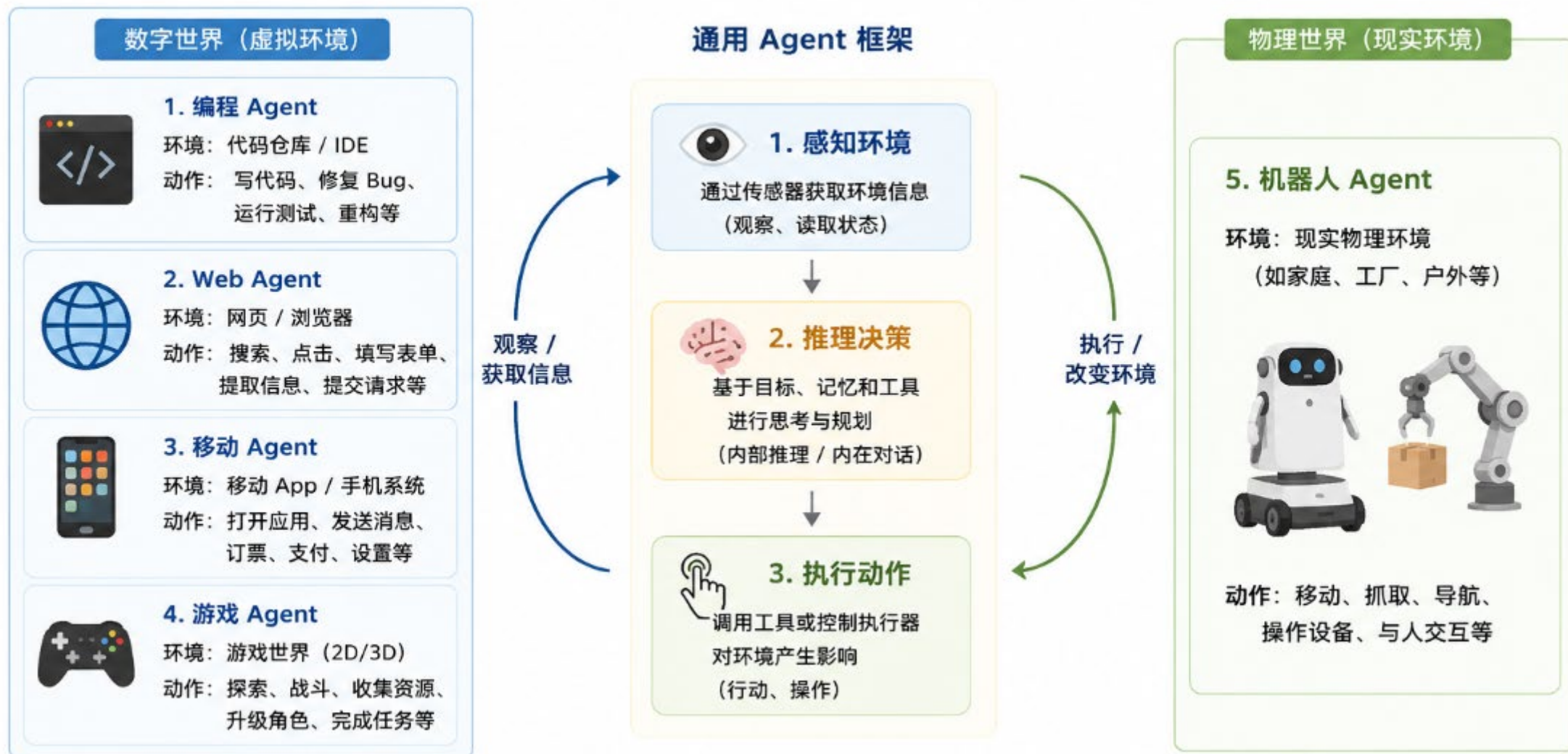
- 对于输入 x , 采样一个位置 i 及其对应的 API 调用候选项 c_i^1 、 c_i^2 、... c_i^k 。
- 然后, 我们执行这些 API 调用, 并过滤掉所有无法降低损失 L_i 的调用。
- 所有剩余的API调用将与原始文本交错排列, 从而生成新的文本 x^*



6. 不同智能体应用、数据与评估

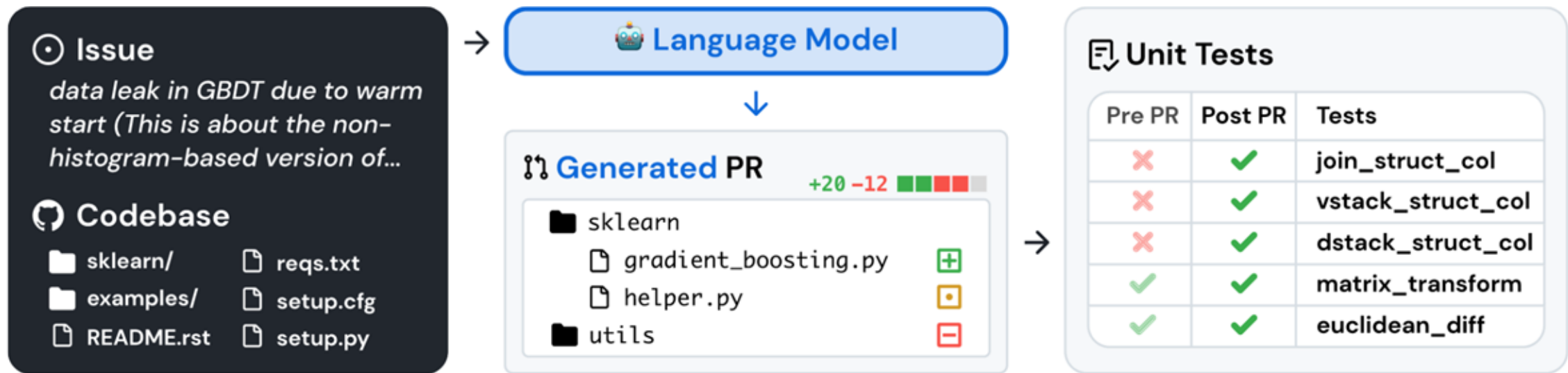


基于环境和领域的Agent应用



编码Agent

- 环境：项目代码仓库、文件系统、IDE.....
- 观察空间：代码文件、可执行文件输出、文档、错误、提交历史.....
- 动作空间：代码编辑、文件搜索/查看、测试更新.....



SWE-bench ([Jimenez & Yang, 2023](#))

Web/App Agent

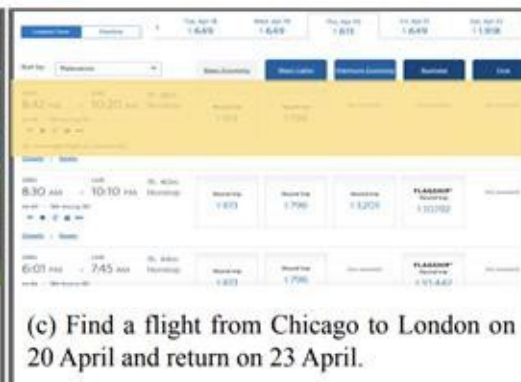
- 环境：网页浏览器/应用程序
- 观察空间：屏幕截图、DOM 树、HTML、历史操作.....
- 动作空间：浏览器/应用程序控件（例如：点击、输入、滚动、拖动、悬停.....）



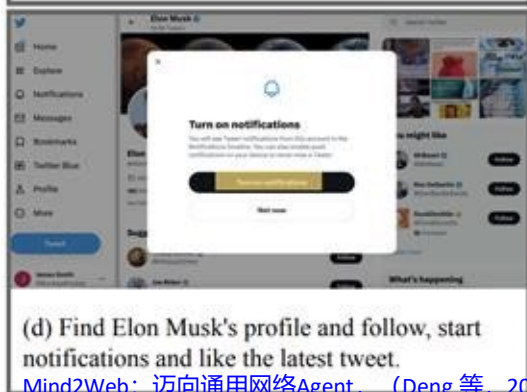
(a) Find one-way flights from New York to Toronto.



(b) Book a roundtrip on July 1 from Mumbai to London and vice versa on July 5 for two adults.



(c) Find a flight from Chicago to London on 20 April and return on 23 April.



(d) Find Elon Musk's profile and follow, start notifications and like the latest tweet.



(e) Browse comedy films streaming on Netflix that was released from 1992 to 2007.



(f) Open page to schedule an appointment for car knowledge test.

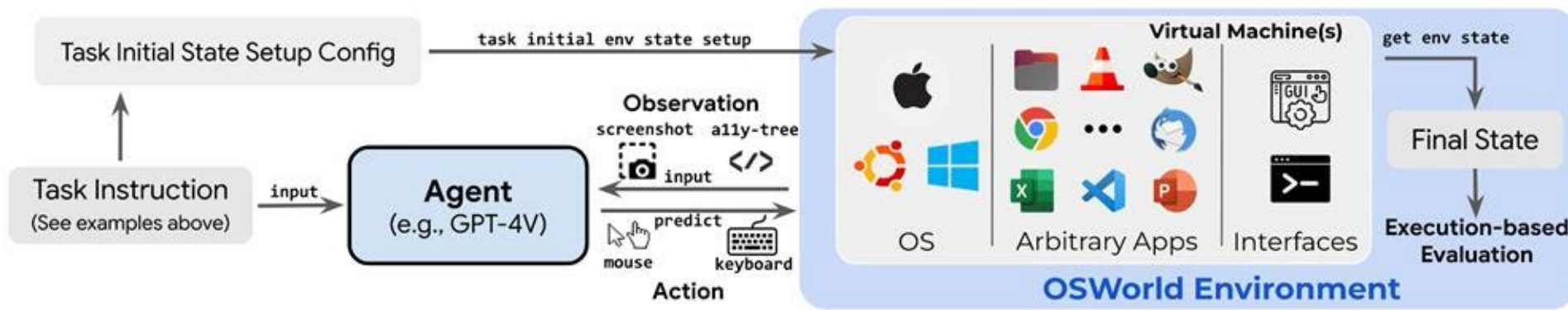
Mind2Web: 迈向通用网络Agent, (Deng 等, 2023)

WebArena: 构建自主Agent的现实主义Web环境, (Zhou等人, 2023)

计算机使用Agent

- 环境：桌面操作系统
- 观察空间：桌面截图、无障碍树、历史操作.....
- 动作空间：键盘/鼠标操作（例如：点击、输入、拖拽、快捷键）

Task instruction 1: Update the bookkeeping sheet with my recent transactions over the past few days in the provided folder.



Agent评估

LLM（普通问答）评估：相对简单



Q: 法国的首都是哪里？

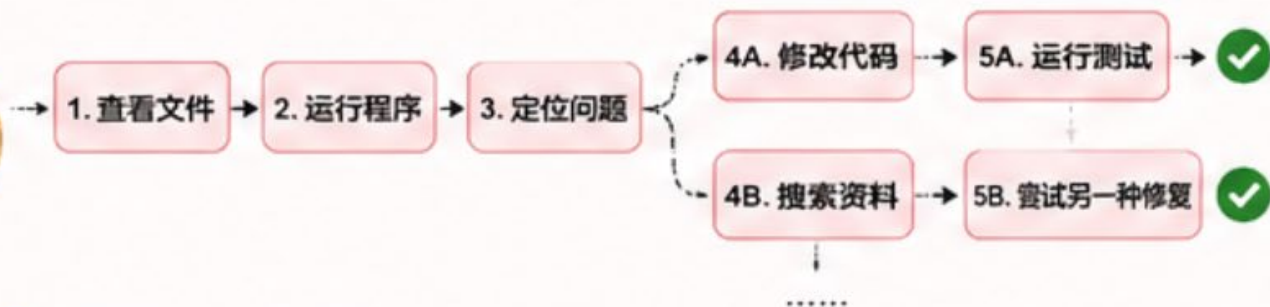
A: 巴黎。

- ✓ 有标准答案
- ✓ 可以自动评分（准确率、ROUGE、BLEU等）
- ✓ 评估重点：答案是否正确

VS

Agent 评估：非常复杂

任务：修复 main.py 的错误



- ✗ 没有唯一正确路径
- ✗ 不同步骤顺序、不同工具都可能成功
- ✗ 需要评估：计划、工具使用、效率、安全性、最终结果等
- ✗ 部分失败可能是环境或工具问题，不一定是 Agent 能力问题

Agent评估

5. 常见评估维度 (Agent)



任务成功率
是否达成目标

完成 / 未完成



步骤效率
用了多少步 / 时间

步骤数、耗时



工具使用
是否选择了正确的工具

精确率、召回率



安全性
是否有危险操作

是否越权、是否破坏环境



鲁棒性
面对噪声、错误是否能恢复

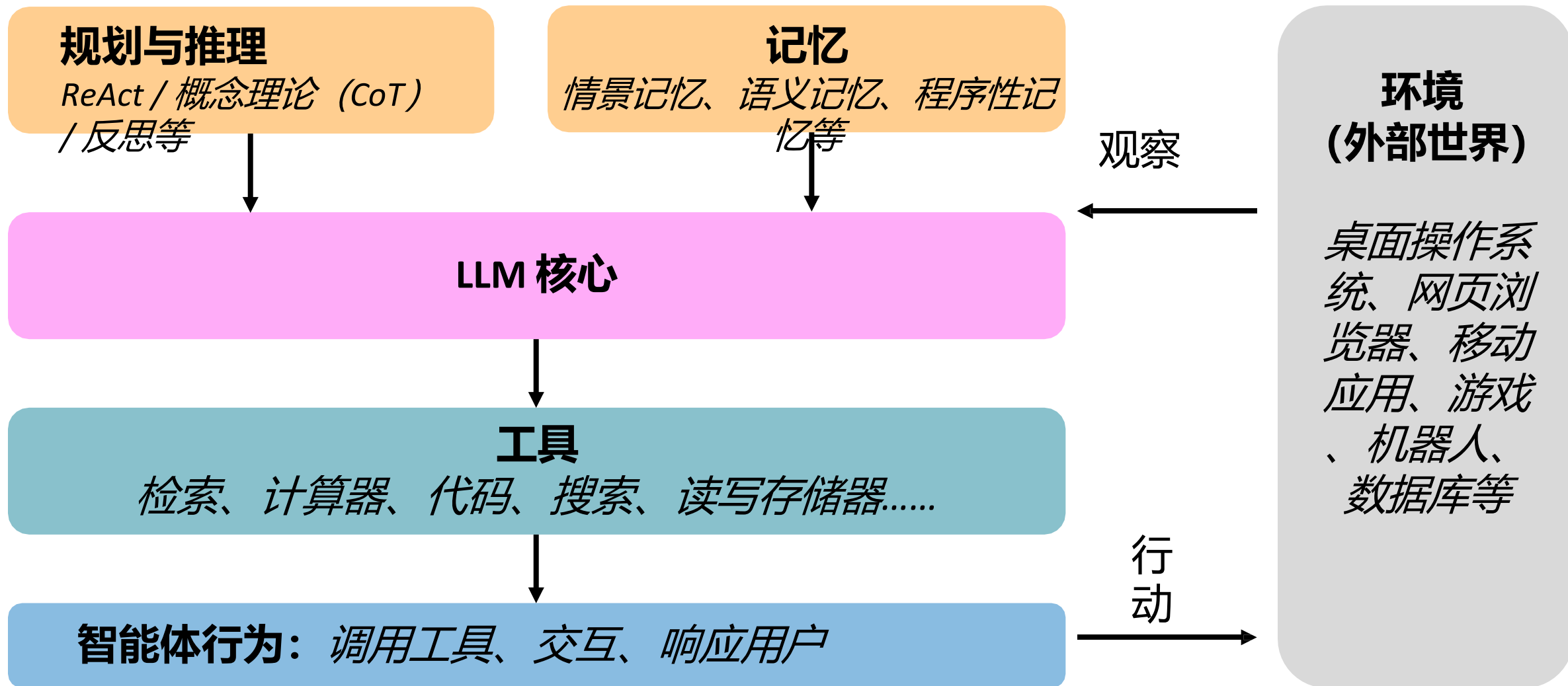
重试、恢复情况



用户满意度
最终结果是否满足需求

人工打分 / 众包

语言Agent：从语言到行动



构建编程Agent (分步指南)

最简单的AI系瞄Agent兵宝尤斜路丧够得烦

Agent = 推理 + 行动 + 环境反馈循环

给定一个任务，例如“为我创建一个网页应用”，我们启动以下循环：

1. 语言模型 (LM) 提出一个操作
2. 你在代码环境中执行该操作
(例如，bash 终端、Docker 容器、Python 解释器)
3. 将输出结果告知语言模型
4. 然后重复步骤 2-4!



John Yang

Thought & Action

Our reproduction script confirms the reported issue: Max & Min are not being converted to R. Let's search for files related to R code generation.

```
$ search_dir "rcode"
```

Observation

Found 16 matches for "rcode":
reproduce_issue.py (3 matches)
sympy/printing/lambdarepr.py (1 matches)
sympy/printing/rcode.py (12 matches)

Thought & Action

The responsible file is likely rcode.py. We should open and inspect this file.

```
$ open sympy/printing/rcode.py
```

Observation

```
[File: sympy/printing/rcode.py (412 lines)]  
1: """The RCodePrinter converts sympy ...  
2: from sympy.core import S  
...  
20: # dictionary of sympy to C functions  
21: known_functions = {  
22:     "Abs": "abs",  
23:     "gamma": "gamma",
```

构建编程Agent（分步指南）

最简单的AI编程Agent其实就是一个大循环。以下是伪代码：

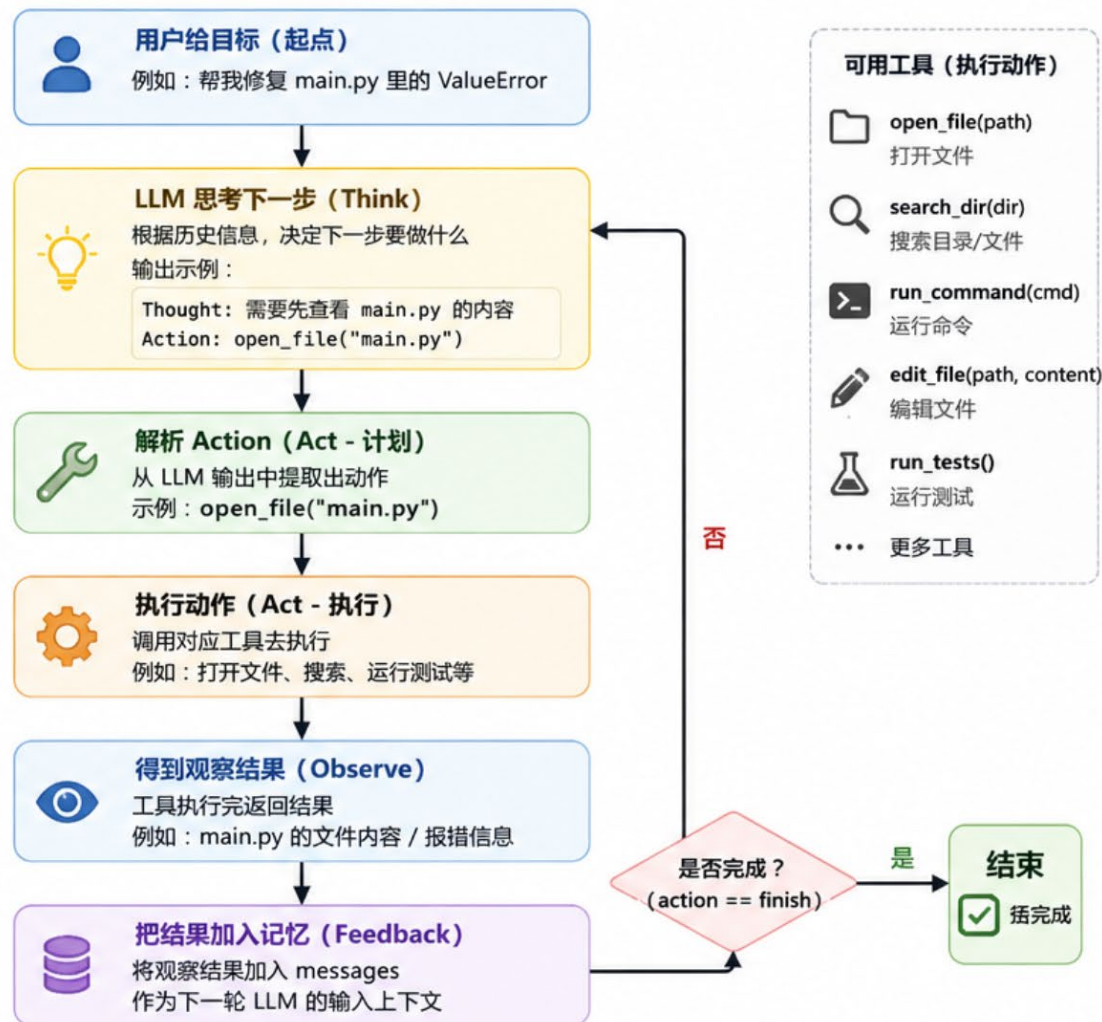
伪代码（Python 风格）

```
1 messages = [  
2     {"role": "user", "content": "帮我修复 main.py 里的 ValueError"}  
3 ]  
4  
5 while True:  
6     # 1. 思考：让 LLM 根据历史信息决定下一步  
7     lm_output = query_lm(messages)  
8  
9     # 2. 记录：保存模型刚才的思考和计划  
10    messages.append({"role": "assistant", "content": lm_output})  
11  
12    # 3. 解析：从 LLM 输出中提取动作  
13    action = parse_action(lm_output)  
14  
15    # 4. 判断是否结束  
16    if action.name == "finish":  
17        break  
18  
19    # 5. 执行：调用工具  
20    observation = execute_action(action)  
21  
22    # 6. 反馈：把工具结果再喂回 LLM  
23    messages.append({"role": "user", "content": observation})
```

关键组件说明

query_lm(messages)	调用大语言模型，根据历史对话生成下一步思考和行动计划
parse_action(output)	解析 LLM 输出，提取出要执行的动作（工具调用）
execute_action(action)	执行动作（如打开文件、搜索代码、运行命令等），并返回结果
messages	对话记忆，保存所有历史，包括思考、行动和观察结果

循环流程图（Think → Act → Observe → Repeat）



构建编程Agent (分步指南)

让我们实现以下三个功能：query_lm； parse_action； execute_action

query_lm： 让 LLM 思考

输入：消息

输出：语言模型的响应

parse_action： 从自然语言提取“动作”

输入：语言模型的响应

输出：bash 操作

execute_action： 真正执行命令

输入：基础操作

输出：标准输出

```
from openai import OpenAI

client = OpenAI(
    api_key="your-api-key-here"
) # or set OPENAI_API_KEY env var

def query_lm(messages):
    response = client.responses.create(
        model="gpt-5.1",
        input=messages
    )
    return response.output_text
```

```
import re

def parse_action(lm_output: str) -> str:
    """Take LM output, return action"""
    matches = re.findall(
        r"<bash_action>(.*?)</bash_action>",
        lm_output,
        re.DOTALL
    )
    return matches[0].strip() if matches else ""
```

```
import subprocess
import os

def execute_action(command: str) -> str:
    """Execute action, return output"""
    result = subprocess.run(
        command,
        shell=True,
        text=True,
        env=os.environ,
        encoding="utf-8",
        errors="replace",
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        timeout=30,
    )
    return result.stdout
```

完整教程请见：<https://minimal-agent.com/>